YENEPOYA
(DEEMED TO BE UNIVERSITY)

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT
A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY)

AN INTERNSHIP REPORT ON

EMOVISION: AI-POWERED REAL-TIME FACIAL EMOTION DETECTION SYSTEM

SUBMITTED BY

ANAND V P

23BBCAMR074

VI SEMESTER

BCA (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

UNDER THE GUIDANCE OF

MS. NAGAVARALAKSHMI

Assistant Professor,

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF

THE DEGREE OF BACHELOR IN COMPUTER APPLICATIONS

April 2026

YENEPOYA
(DEEMED TO BE UNIVERSITY)

CERTIFICATE

This is to certify that **Mr.ANAND V P**, bearing Registration Number **23BBCAMR074**, of Batch **2023–2026**, is a bonafide student of the Bachelor of Computer Applications (BCA) programme at Yenepoya Institute of Arts, Science, Commerce and Management, affiliated to Yenepoya (Deemed to be University).

He has successfully completed the internship titled **EMOVISION:AI-Powered Real-Time Facial Emotion Detection System** at **IBM, Bhartiya City (Bengaluru)** under the guidance of **Ms.Nagavaralakshmi.**

This internship report is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications at Yenepoya (Deemed to be University), Bengaluru, Karnataka.

Signature of Internal Guide **Signature of HOD**

Signature of Vice Principal **Signature of Director-Academics**

YENEPOYA
(DEEMED TO BE UNIVERSITY)

DECLARATION

I, **Mr.ANAND V P** bearing Registration Number **23BBCAMR074** hereby declare that the internship report titled **EMOVISION:AI-Powered Real-Time Facial Emotion Detection System**, with reference to **IBM,** has been prepared by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) at Yenepoya (Deemed to be University).

I further declare that this internship report is the result of my own work carried out under the guidance of **Ms.Nagavaralakshmi, Assistant Professor**, Yenepoya Institute of Arts, Science, Commerce and Management.

I also confirm that this report has not been previously submitted to any university or institution for the award of any degree or diploma.

Place:

Date:

Name : **ANAND V P**

Reg No : **23BBCAMR074**

Campus ID : **30561**

YENEPOYA
(DEEMED TO BE UNIVERSITY)

ACKNOWLEDGEMENT

Firstly, I Would like to express my immense gratitude towards my institution **Yenepoya Deemed to be University** which created a great platform to attain profound technical skills in the fields of Computer Applications, there by fulfilling my most cherished goal.

I am thankful to **Mr. Moideen Khurshid Yenepoya** (Managing Director) YDU, Bengaluru Campus and Management for providing the necessary infrastructure and labs.

I feel privileged to offer my sincere thanks and deep sense of gratitude to **Dr. Jerry John** (Director-Academics), **Dr. Melita L** (Vice Principal, YIASCM, Bengaluru Campus), **Mr. Syed Umar Abbasi** (HOD, Department of Computer Science & Information Technology, Bengaluru Campus), **Dr. R Kaviyarasi** (Academic Head, Department of Computer Science & Information Technology, Bengaluru Campus) for extending their co-operation in doing this project.

I express my sincere gratitude and thankfulness towards my project guide, **Ms.Nagavaralakshmi, Assistant Professor,** for her/his valuable time and guidance throughout the BCA project.

My special thanks to all the faculty members of the Department of Computer Science & Information Technology for their valuable advises at every stage.

I would like to express my deep sense and earnest thanks giving to my dear Parents and my family members for their moral support and kind cooperation in doing the project.

I would also like to thank all my friends, whose direct or indirect help has enabled me to complete this project.

Thank you all once again for your invaluable support.

Place:

Date:

Name : **ANAND V P**

Campus ID: **30561**

Reg No : **23BBCAMR074**

TEAM MEMBERS & CONTRIBUTIONS

EmoVision was developed as a collaborative team effort by five members of the Bachelor of Computer Applications (BCA — AI & ML) programme at Yenepoya Institute of Arts, Science, Commerce and Management, Bengaluru Campus, during their internship at IBM, Bhartiya City, Bangalore, from January to June 2026. While each team member brought distinct expertise to the project, the work was conducted in an integrated and cooperative manner, with continuous cross-functional collaboration ensuring the coherence of the final system.

S. No.	Name	Reg. No.	Role	Primary Responsibilities
1	ANAND V P	23BBCAMR 074	Project Lead & ML Architect	Overall system architecture and design decisions. Full-stack development of the FastAPI backend, WebSocket streaming engine, and ML inference pipeline. EfficientNet-B0 model training, EWC continual learning implementation, ONNX ensemble integration, temporal smoothing, and confidence gate. GitHub repository setup and Docker deployment. Primary technical writer.
2	CLEVIN AJU	23BBCAMR 116	Frontend Developer & UI/UX Designer	Design and development of the EmoVision Single Page Application (index.html). Landing page design, image upload mode, live camera mode, and Chart.js analytics dashboard. CSS theming, responsive layout, and accessibility review. Coordination of visual design guidelines and user experience evaluation.
3	ABHINAV C P	23BBCAMR 023	Data Engineer & Dataset Manager	Collection, preprocessing, and organisation of the FER2013 and RAF-DB training datasets. Auto-data-collection pipeline configuration. Responsible for collecting Indian face training samples for continual fine-tuning. Kaggle API setup, dataset balancing analysis, and management of the collected_data directory.
4	ADHWAITH K G	23BBCAMR 030	QA Engineer & Test Lead	Design and execution of the complete test suite: unit tests (25 cases), integration tests (10 cases), and system tests (14 cases). Performance benchmarking and FPS measurement. Documentation of test results, confusion matrix analysis, and defect tracking throughout development sprints.
5	MELETHIL MUHAMMED LAHAN	23BBCAMR 190	Research Analyst & Technical Writer	Literature survey research across 10+ foundational papers (DeepFace, EfficientNet, RetinaFace, EWC, FER2013, RAF-DB). Comparative analysis of existing emotion recognition systems. Assistance with report writing, bibliography compilation (APA format), and preparation of the project presentation slides.

Table: Team Members and Their Roles in the EmoVision Project

ABSTRACT

EmoVision is a comprehensive, production-grade, real-time facial emotion detection system developed as part of a six-month internship programme with IBM, Bhartiya City, Bangalore. The system leverages state-of-the-art deep learning methodologies to automatically detect and classify human facial emotions from both static images and live video streams, operating entirely on local hardware without any dependency on cloud-based services.

The primary objective of EmoVision is to provide an accessible, accurate, and privacy-preserving emotion recognition platform capable of detecting seven distinct emotional categories — anger, disgust, fear, happiness, sadness, surprise, and neutral expressions — across multiple faces simultaneously in real time. The system is built upon a modern three-tier architecture comprising a professional web-based Single Page Application frontend, a scalable FastAPI backend supporting both REST and WebSocket communication protocols, and a sophisticated machine learning inference engine.

The ML inference pipeline employs a dual-backend architecture using DeepFace with RetinaFace as the primary face detection and emotion classification engine, complemented by a custom-trained EfficientNet-B0 model fine-tuned on the FER2013 dataset comprising 35,887 labelled facial expression images. The ensemble approach blends predictions from both models, achieving superior robustness compared to either model in isolation. Advanced post-processing techniques including temporal smoothing over a five-frame rolling window, IoU-based face tracking, and a confidence gate mechanism significantly reduce prediction jitter in live streaming scenarios.

The system achieves an emotion classification accuracy of 66–68% on the FER2013 benchmark dataset, matching documented human-level performance on this notoriously challenging dataset. Live camera inference operates at 8–10 frames per second on standard CPU hardware, rising to over 25 frames per second with GPU acceleration. A continual learning module employing Elastic Weight Consolidation (EWC) enables the model to improve incrementally on domain-specific data — including Indian faces — without catastrophically forgetting its pre-trained knowledge.

The project demonstrates that sophisticated AI systems can be made practically deployable, user-friendly, and accessible without sacrificing technical rigour. EmoVision holds significant potential for applications in human-computer interaction, mental health monitoring, educational technology, market research, and interactive entertainment. This report details the complete system architecture, implementation methodology, testing strategy, performance metrics, and future enhancement roadmap for the EmoVision emotion detection system.

TABLE OF CONTENTS

Title Page	1
Certificate	2
Declaration	3
Acknowledgement	4
Team Members & Contributions	5
Abstract	6
Table of Contents	7
List of Figures	9
List of Tables	10
List of Abbreviations	11
CHAPTER 1: INTRODUCTION	13
1.1 Background and Context	13
1.2 Problem Statement	14
1.3 Objectives of the System	15
1.4 Importance and Motivation	16
1.5 Scope of the System	17
1.6 Organisation of the Report	18
1.7 Team Roles and Contributions	19
CHAPTER 2: LITERATURE SURVEY	21
2.1 Introduction to Facial Expression Recognition	21
2.2 Review of Foundational Research Papers	22
2.3 Comparison of Existing Systems	26
2.4 Dataset Landscape	27
2.5 Gaps Identified	28
CHAPTER 3: SPECIFICATIONS & SYSTEM DESIGN	30
3.1 Hardware Requirements	30
3.2 Software Requirements	31
3.3 System Architecture	32
3.4 Data Flow Diagram	35
3.5 UML Diagrams	37
3.6 ML Pipeline Design	42
3.7 Database and Data Store Design	44
CHAPTER 4: IMPLEMENTATION	45
4.1 Development Environment Setup	45
4.2 Backend Implementation	45
4.3 ML Pipeline Implementation	48

4.4 Custom Model Training Implementation	52
4.5 Continual Learning Implementation (EWC)	54
4.6 Frontend Implementation	56
4.7 Docker Deployment Implementation	58
4.8 GitHub Repository Setup	59
CHAPTER 5: TESTING	61
5.1 Testing Philosophy and Strategy	61
5.2 Unit Testing	61
5.3 Integration Testing	63
5.4 System Testing	64
5.5 Performance Testing	66
5.6 Model Accuracy Evaluation	67
5.7 User Acceptance Testing (UAT)	68
CHAPTER 6: FEATURE ENHANCEMENTS	72
CHAPTER 7: CONCLUSION	77
BIBLIOGRAPHY	81
APPENDIX A: PROJECT SCREENSHOTS	83
APPENDIX B: PROJECT TIMELINE	90
APPENDIX C: CONCEPT ILLUSTRATIONS	93

LIST OF FIGURES

Figure Number	Figure Title	Page Number
Figure 3.1	EmoVision Three-Tier System Architecture Diagram	34
Figure 3.2	Level 0 Context DFD – EmoVision System	36
Figure 3.3	Level 1 DFD – EmoVision Processing Pipeline	37
Figure 3.4	Use Case Diagram – EmoVision Actors and Functions	38
Figure 3.5	Class Diagram – EmoVision Core Components	39
Figure 3.6	Sequence Diagram – Image Analysis Request	40
Figure 3.7	Sequence Diagram – Live Camera WebSocket Streaming	41
Figure 3.8	Activity Diagram – EmoVision User Interaction Flow	42
Figure 4.1	FastAPI Main Application - Image Analysis Endpoint	47
Figure 4.2	WebSocket Handler – Real-Time Camera Streaming	48
Figure 4.3	Emotion Pipeline Backend Initialisation and Warm-up	49
Figure 4.4	Deep Face Analysis with NumPy Type Safety Casting	51
Figure 4.5	FaceSmoother IoU Tracking and Rolling Average	52
Figure 4.6	EfficientNet-B0 Architecture with Custom Classification Head	53
Figure 4.7	Class Imbalance Correction Mechanisms	54
Figure 4.8	EWC Implementation - Fisher Information Matrix	55
Figure 4.9	Canvas Overlay – X-Coordinate Mirroring Solution	58
Figure 4.10	Dockerfile – EmoVision Container Specification	59
Figure A.1	EmoVision Landing Page	83
Figure A.2	Image Upload Mode - Analysis Results	84
Figure A.3	Multi-Face Detection Analysis	86
Figure A.4	Live Camera Mode - Real-Time Detection	87
Figure A.5	Session Analytics Dashboard	88
Figure A.6	EmoVision GitHub Repository	89
Figure B.1	High-Level Project Timeline (Gantt Overview)	92
Figure C.1	Seven Facial Expression Categories	93
Figure C.2	Deep Learning Inference Pipeline Concept	94
Figure C.3	Real-World Application Domains	94
Figure C.4	Elastic Weight Consolidation Concept	95

LIST OF TABLES

Table Number	Table Title	Page Number
Table 1.1	System Overview Summary	19
Table 2.1	Comparative Analysis of Emotion Recognition Systems	27
Table 3.1	Hardware Requirements Specification	30
Table 3.2	Expected Performance by Hardware Configuration	31
Table 3.3	Software Requirements Specification	31
Table 3.4	ML Pipeline Design Parameters	43
Table 4.1	Key Implementation Files and Their Purpose	59
Table 5.1	Unit Test Cases – ML Pipeline	61
Table 5.2	Unit Test Cases – FastAPI Backend	62
Table 5.3	Integration Test Cases	63
Table 5.4	System Test Cases	65
Table 5.5	Performance Test Results	66
Table 5.6	Model Accuracy on FER2013 PrivateTest Set	67
Table 5.7	Confusion Matrix – FER2013 PrivateTest Set	68
Table 5.8	User Acceptance Test Task Completion Results	68
Table 5.9	UAT Accuracy Assessment on Indian Faces	70
Table 6.1	Feature Enhancement Roadmap	75
Table B.1	EmoVision Project Timeline (Planned vs Actual)	90
Table B.2	Sprint Velocity	91

LIST OF ABBREVIATIONS

Table: List of Abbreviations Used in this Report

S.No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	DL	Deep Learning
4	CNN	Convolutional Neural Network
5	API	Application Programming Interface
6	REST	Representational State Transfer
7	WS	WebSocket
8	HTTP	HyperText Transfer Protocol
9	HTML	HyperText Markup Language
10	CSS	Cascading Style Sheets
11	JS	JavaScript
12	SPA	Single Page Application
13	JSON	JavaScript Object Notation
14	ONNX	Open Neural Network Exchange
15	FER	Facial Expression Recognition
16	CPU	Central Processing Unit
17	GPU	Graphics Processing Unit
18	FPS	Frames Per Second
19	UI	User Interface
20	UX	User Experience
21	EWC	Elastic Weight Consolidation
22	IoU	Intersection over Union

S.No.	Abbreviation	Full Form
23	ASGI	Asynchronous Server Gateway Interface
24	CORS	Cross-Origin Resource Sharing
25	BCA	Bachelor of Computer Applications
26	YDU	Yenepoya Deemed to be University
27	IBM	International Business Machines
28	UML	Unified Modelling Language
29	DFD	Data Flow Diagram
30	RAF-DB	Real-world Affective Faces Database
31	HCI	Human-Computer Interaction
32	MTCNN	Multi-Task Cascaded Convolutional Networks
33	LR	Learning Rate
34	JPEG	Joint Photographic Experts Group
35	PNG	Portable Network Graphics

CHAPTER 1: INTRODUCTION

1.1 Background and Context

The human face is one of the most expressive communication channels available to our species. Long before language evolved to its current sophistication, our ancestors read intent, danger, affection, and warning in the subtle movements of brows, lips, and eyes. This biological inheritance means that facial expressions carry a richness of meaning that words alone cannot always replicate. In contemporary society, the ability to automatically interpret these expressions has become a compelling and commercially significant research frontier, positioned at the intersection of computer vision, cognitive science, and deep learning.

Affective computing — the field dedicated to enabling machines to recognise, interpret, and respond to human emotional states — has expanded rapidly since Rosalind Picard first articulated its scientific foundations in 1997. The practical applications of such capabilities are vast: from detecting driver fatigue and preventing road accidents, to supporting therapists with objective emotion monitoring tools, to enabling retailers to measure genuine consumer reactions to product displays in real time. Yet despite decades of research and significant commercial investment, the deployment of emotion recognition technology has historically been constrained by several persistent limitations: poor accuracy on demographically diverse populations, dependence on expensive specialised hardware, inability to process multiple faces simultaneously, and deep reliance on cloud-based infrastructure that raises legitimate privacy and data sovereignty concerns.

The rapid democratisation of deep learning — driven by open-source frameworks such as PyTorch and TensorFlow, publicly available large-scale datasets like FER2013 and AffectNet, and pre-trained model libraries — has fundamentally altered this landscape. Today, a motivated engineer with access to a standard laptop can implement and deploy an emotion recognition pipeline of a quality that would have required a specialised research laboratory just a decade ago. EmoVision is precisely such a system: a thoughtfully engineered, full-stack emotion detection platform that combines state-of-the-art computer vision with a clean, production-ready software architecture — built, tested, and documented during a six-month internship at IBM India, Bhartiya City, Bangalore.

The Bhartiya City campus presents a particularly interesting context for this work. Co-located alongside the IBM development centre is the off-campus facility of Yenepoya Deemed to be University, creating a rare adjacency between academic learning and professional engineering practice. The internship programme designed within this context was structured to bridge that gap deliberately: interns were expected not merely to complete assigned tasks but to design, build, and document a complete AI-driven software system to professional standards. EmoVision is the tangible outcome of that challenge.

1.2 Problem Statement

Despite significant progress in emotion recognition research, practical deployments continue to suffer from a constellation of unresolved problems that limit real-world utility. An analysis of the existing landscape reveals the following core challenges:

- Privacy and Data Security: The dominant commercial emotion recognition systems — offered by providers such as Amazon Rekognition, Microsoft Azure Face API, and Google Vision AI — are cloud-dependent. Every image or video frame must be transmitted to a remote server for processing. For applications in healthcare, education, security, or any context involving vulnerable individuals, this represents an unacceptable privacy risk. Data may be stored, repurposed, or exposed through breaches without the subject's knowledge or consent.
- Demographic Bias: Foundational training datasets for emotion recognition, including the widely-used FER2013 dataset, are heavily skewed towards Western and East Asian face populations. Models trained exclusively on such data perform significantly worse on South Asian, African, and other underrepresented demographic groups. This is not merely an academic concern — it represents a genuine equity issue when such systems are deployed in countries like India without correction.
- Real-Time Limitations: Many existing systems are optimised for batch processing of pre-recorded video rather than genuine real-time inference. Achieving 8–15 frames per second on consumer CPU hardware, without GPU acceleration, requires careful architectural choices that most academic implementations do not address.
- System Accessibility: High-quality emotion detection libraries are either proprietary and prohibitively expensive, or open-source but require substantial engineering expertise to deploy. There is a clear gap for a system that is simultaneously sophisticated, accessible, and practically deployable by non-specialists.
- Lack of Continual Improvement: Most deployed models are static — trained once and frozen. As the operational environment shifts (new users, different lighting, different demographics), accuracy degrades silently. A self-improving system that can learn from its deployment environment without losing its initial training represents a significant advance.

EmoVision is designed to address each of these challenges systematically: local inference preserves privacy, a continual learning pipeline addresses demographic bias incrementally, a WebSocket-based architecture enables genuine real-time performance, and a zero-build-step frontend makes the system immediately deployable for non-expert users.

1.3 Objectives of the System

The objectives of EmoVision were established at the outset of the internship through a structured requirements analysis process and refined iteratively through consultation with the internal guide and IBM mentors. The objectives are organised into primary and secondary tiers to reflect their relative priority.

1.3.1 Primary Objectives

1. Develop a real-time, multi-face facial emotion detection system capable of classifying seven distinct emotional categories — anger, disgust, fear, happiness, sadness, surprise, and neutral — with a minimum accuracy of 65% on the FER2013 benchmark dataset.
2. Achieve live camera inference at a sustained rate of 8 frames per second or greater on standard consumer CPU hardware, without requiring GPU acceleration, making the system practically accessible to the broadest possible user base.
3. Implement a full-stack web application architecture with a professional, production-ready user interface accessible via any modern web browser, requiring no installation or configuration from the end user beyond running a single Python command.
4. Ensure complete local operation without any cloud dependency, so that no user data — images or video frames — is ever transmitted to external servers, fully preserving the privacy and data sovereignty of system users.
5. Implement a continual learning pipeline using Elastic Weight Consolidation (EWC) that allows the model to be iteratively improved on domain-specific data — particularly Indian faces — without catastrophically forgetting the knowledge encoded during initial pre-training.

1.3.2 Secondary Objectives

6. Provide comprehensive session analytics including emotion timeline visualisation, session history charts, face-specific confidence scores, and real-time inference metrics accessible through an integrated dashboard.
7. Support simultaneous multi-face detection and independent classification in a single frame, with unique colour-coded visual tracking for each detected face.
8. Export the trained model to the ONNX format for deployment-optimised inference, achieving a 5–10x speedup over PyTorch inference on CPU through graph-level optimisation.
9. Package the complete system within a Docker container for one-command cloud or local deployment, enabling easy reproduction and sharing.

10. Produce a professional, comprehensive internship report suitable for academic archival and publication, documenting the complete technical design, implementation, testing, and evaluation of the system.

1.4 Importance and Motivation

The importance of EmoVision extends across several dimensions — technical, social, and academic — and the motivations for undertaking this project were equally multifaceted.

From a technical perspective, EmoVision represents a genuine engineering challenge. Building a system that correctly handles face detection under varying lighting and pose, classifies emotions with acceptable accuracy in real-time, maintains stable face tracking across frames, and presents all of this through a polished web interface requires deep integration of computer vision, backend engineering, and frontend development — a rare combination in a single project.

From a social perspective, the implications of emotion AI are significant. When implemented responsibly — with privacy protection, transparency, and fairness controls — emotion recognition can meaningfully improve human wellbeing. Mental health professionals working with non-verbal patients, educators monitoring student engagement in remote learning environments, and accessibility engineers building more responsive assistive technologies can all benefit from this capability. EmoVision's local-first architecture is a deliberate design choice that prioritises these positive applications while mitigating the surveillance risks associated with cloud-dependent alternatives.

From a personal and academic perspective, the project represented an opportunity to work at the frontier of applied AI during a formative stage of professional development. The IBM internship context brought added rigour: professional code review, documentation standards, and the expectation of deployment-ready output rather than research prototypes. This context elevated the quality of every engineering decision made throughout the project's lifecycle.

The project also draws attention to an underserved research need: improving AI system accuracy on Indian faces. With a population of over 1.4 billion people, India represents the single largest potential user base for any global AI application, yet the models that power most commercial systems have been trained disproportionately on Western and East Asian data. EmoVision's continual learning pipeline, automatic data collection feature, and Indian-face fine-tuning capability represent concrete, practical steps toward addressing this inequity.

1.5 Scope of the System

The scope of EmoVision encompasses the complete lifecycle of a production-grade AI application, from data processing and model training through to web-based deployment and continual learning. Specifically, the system encompasses the following capabilities and functional boundaries.

1.5.1 What the System Covers

- Image Analysis Mode: Users can upload JPEG, PNG, or WebP images of any resolution up to 20 MB. The system detects all faces in the image, classifies the dominant emotion for each face with a full probability distribution across seven emotion categories, and returns a professionally annotated version of the image with bounding boxes, emotion labels, and confidence percentages.
- Live Camera Mode: Users can activate their device webcam and receive real-time emotion analysis via a persistent WebSocket connection. The system captures frames at approximately 8 frames per second, analyses each frame for face presence and emotion, and overlays the results directly onto the live video feed using the HTML5 Canvas API.
- Multi-Face Detection and Tracking: The system detects and independently analyses all faces present in a given frame simultaneously. Each detected face is assigned a unique colour-coded identifier that persists across frames through IoU-based tracking, enabling consistent analysis even in group scenarios.
- Session Analytics Dashboard: The frontend provides real-time analytics including an emotion timeline chart, session history doughnut chart, inference latency metrics, frames-per-second counter, and per-face confidence breakdowns.
- Model Training and Fine-Tuning: The system includes a complete model training pipeline (train.py) for training EfficientNet-B0 from scratch or fine-tuning on custom datasets, and a continual learning trainer (continual_train.py) for domain-specific improvement using EWC.
- Automatic Data Collection: When the data collection mode is enabled, the live inference pipeline automatically saves high-confidence face crops to a structured folder hierarchy for use in future training cycles, enabling a self-improving feedback loop.
- ONNX Export and Ensemble Inference: The custom-trained model can be exported to ONNX format and loaded alongside DeepFace for ensemble inference, blending the predictions of both models for improved accuracy.
- Docker Deployment: The complete system can be packaged and deployed using a single Docker command, making it suitable for cloud hosting, institutional deployment, or offline demonstration.

1.5.2 Limitations and Boundaries

- The system is optimised for frontal or near-frontal face detection. Significantly rotated faces (beyond approximately 45 degrees from frontal) may not be detected or may yield lower accuracy classifications.
- Classification accuracy on the FER2013 benchmark dataset is 66–68%, matching human-level performance on this specific dataset but acknowledging that the dataset itself contains label noise and limited demographic diversity.
- Live camera inference is limited to 8–10 frames per second on standard CPU hardware. GPU acceleration (CUDA-enabled) can push this to 25 frames per second or beyond but is not required for functional operation.
- The system does not currently include voice or speech emotion recognition; it operates exclusively on visual (facial) information.
- The system does not perform age estimation, gender classification, or identity recognition — it is limited to emotion classification from facial expressions.
- Extreme lighting conditions (very low light, harsh backlighting) can reduce detection accuracy. Adequate ambient lighting is recommended for optimal performance.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 presents a comprehensive literature survey covering foundational and contemporary research in facial expression recognition, deep learning architectures for emotion classification, and relevant datasets. Chapter 3 details the system design, including architectural diagrams, UML models, data flow diagrams, and the rationale behind key design decisions. Chapter 4 describes the implementation in technical depth, covering the backend, ML pipeline, frontend, and deployment infrastructure, with relevant code excerpts. Chapter 5 presents the testing strategy and results, including unit, integration, and system-level test cases. Chapter 6 discusses feature enhancements and the future roadmap. Chapter 7 presents the conclusion and Chapter 8 provides the complete bibliography.

Table 1.1: System Overview Summary

Parameter	Value / Description
System Name	EmoVision
System Type	Full-Stack AI Web Application
Primary Function	Real-time Facial Emotion Detection
Number of Emotion Classes	7 (Anger, Disgust, Fear, Happy, Sad, Surprise, Neutral)
Primary ML Framework	DeepFace + Custom EfficientNet-B0 (PyTorch)
Backend Framework	FastAPI (Python 3.12, ASGI)
Frontend Technology	Vanilla HTML5 / CSS3 / JavaScript (Zero build step)
Real-time Protocol	WebSocket (ws:// / wss://)
Dataset Used (Training)	FER2013 (35,887 images), RAF-DB (29,672 images)
Benchmark Accuracy	66–68% (FER2013 Test Set)
Live Inference Speed	8–10 FPS (CPU), 25+ FPS (GPU)
Maximum Image Size	20 MB (JPEG, PNG, WebP)
Deployment	Local (Python), Docker Container
Privacy Architecture	100% Local — No cloud transmission
Continual Learning	EWC + Rehearsal Buffer
Internship Organisation	IBM India, Bhartiya City, Bangalore
Duration	January 2025 – June 2025 (6 Months)
Status	98% Complete — Active ongoing training improvement

1.7 Team Roles and Contributions

EmoVision was conceived and developed as a team project with clearly defined role boundaries that reflected each member's skill set and academic background. While the system is architecturally unified — a single coherent application with no fragmentation between team members' contributions — the development process was organised around functional ownership of specific system components, ensuring accountability and focused expertise at every layer of the stack.

1.7.1 My Role in the Project (Project Lead and ML Architect)

I(Anand V P) assumed overall responsibility for the technical architecture, implementation, and delivery of the EmoVision system. This role encompassed the widest scope of any team member's contribution and required the deepest integration of knowledge across multiple technical domains. Specific contributions included:

- Defining the complete system architecture — the three-tier FastAPI backend, WebSocket streaming protocol, and ML inference engine — and making all foundational technology selection decisions.
- Implementing the EmotionPipeline class (engine/pipeline.py), including the dual-backend initialisation, DeepFace integration, ONNX ensemble merging, FaceSmoother temporal smoothing, and confidence gate mechanism.
- Designing and implementing the EfficientNet-B0 fine-tuning training pipeline (training/train.py), including the class-weighted loss function, weighted random sampling strategy, label smoothing, cosine annealing LR scheduler, and ONNX export.
- Implementing the EWC (Elastic Weight Consolidation) continual learning trainer (continual_train.py), including Fisher Information Matrix computation and rehearsal buffer mixing — the most technically advanced component of the system.
- Debugging and resolving all major integration issues encountered during development, including the numpy.float32 JSON serialisation incompatibility, the UTF-8 encoding error on Windows, the git history rewrite to remove the accidentally committed venv directory, and the canvas overlay X-coordinate mirroring problem.
- Establishing the GitHub repository structure, .gitignore configuration, Docker containerisation, and all deployment infrastructure.
- Writing the primary technical documentation, README, FINETUNING.md, and this internship report.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction to Facial Expression Recognition

The scientific study of facial expression and its relationship to emotional states has roots that extend well beyond the digital era. Charles Darwin's seminal 1872 work, The Expression of the Emotions in Man and Animals, proposed that facial expressions are universal across human cultures — a thesis that profoundly influenced subsequent research in behavioural science. In the 1970s, Paul Ekman and Wallace Friesen formalised this observation through the development of the Facial Action Coding System (FACS), a comprehensive anatomical taxonomy that describes facial expressions in terms of Action Units (AUs) — discrete muscle movements observable on the face. FACS remains a foundational reference system in affective computing research today, providing the conceptual ground truth against which automated systems are evaluated.

The transition from manual annotation of facial expressions to automated computational recognition began in earnest in the 1990s. Early computer vision approaches relied heavily on hand-crafted features: techniques such as the Gabor Wavelet Transform were used to capture texture information from facial regions, while Local Binary Patterns (LBP) and Histogram of Oriented Gradients (HOG) provided compact, rotation-tolerant descriptors of facial structure. These features were then fed into classical machine learning classifiers — Support Vector Machines, Random Forests, and K-Nearest Neighbour algorithms — to produce emotion labels. While these approaches demonstrated proof-of-concept success on controlled laboratory datasets, they were brittle in the face of real-world variation: changes in lighting, pose, facial hair, and occlusion caused performance to degrade sharply.

The deep learning revolution that began around 2012 with AlexNet's dramatic success on the ImageNet Large Scale Visual Recognition Challenge transformed the facial expression recognition landscape. Convolutional Neural Networks (CNNs), with their capacity to learn hierarchical feature representations directly from raw pixel data, systematically outperformed hand-crafted feature approaches on every major emotion recognition benchmark. By 2015, deep learning-based systems had achieved human-parity on several controlled datasets and were rapidly closing the gap on more challenging in-the-wild benchmarks. The availability of large-scale annotated datasets — FER2013 being the most prominent — and the democratisation of GPU computing through platforms like CUDA created the conditions for an explosion of research that continues to accelerate.

2.2 Review of Foundational Research Papers

2.2.1 DeepFace: Closing the Gap to Human-Level Performance (Taigman et al., 2014)

The DeepFace paper by Taigman, Yang, Ranzato, and Wolf (2014) from Facebook AI Research represents a watershed moment in deep learning-based face analysis. Published at the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), the paper introduced a nine-layer deep CNN trained on a dataset of four million facial images, achieving 97.35% accuracy on the Labeled Faces in the Wild (LFW) benchmark — within 0.28% of human performance. The key architectural contribution was a 3D face alignment step prior to classification, using a 3D face model to frontalize varying pose images before feeding them to the recognition network.

The relevance of DeepFace to EmoVision lies in its demonstration that deep CNNs, when trained on sufficiently large and diverse datasets, can learn face representations that generalise across the enormous natural variation of the human face. The DeepFace library, which evolved from this foundational work, serves as the primary inference backend for EmoVision's production deployment, providing both face detection (through its RetinaFace integration) and emotion classification capabilities.

2.2.2 EfficientNet: Rethinking Model Scaling for CNNs (Tan and Le, 2019)

EfficientNet, proposed by Tan and Le at Google Brain and presented at the International Conference on Machine Learning (ICML) 2019, introduced a principled approach to scaling convolutional neural network architectures. Previous work had typically scaled networks along a single dimension — making them deeper (more layers), wider (more feature maps per layer), or higher-resolution (larger input images) — without systematic consideration of how these dimensions interact. Tan and Le proposed compound scaling: a method that scales all three dimensions simultaneously according to a fixed ratio, guided by a neural architecture search process that identifies the optimal baseline architecture.

The EfficientNet-B0 architecture — the baseline model in the family — achieves comparable accuracy to much larger networks (including ResNet-50 and VGG-16) at a fraction of the parameter count and computational cost. This efficiency-accuracy trade-off makes EfficientNet-B0 particularly well-suited to the constraints of EmoVision's custom training pipeline: it delivers competitive emotion classification accuracy while remaining deployable on CPU hardware without prohibitive memory or compute requirements. In the EmoVision implementation, EfficientNet-B0 is fine-tuned through transfer learning on the FER2013 dataset with the final three feature extraction blocks and the classification head unfrozen, enabling domain-specific adaptation while preserving the generalisable feature representations learned from ImageNet pre-training.

2.2.3 RetinaFace: Single-Stage Dense Face Localisation in the Wild (Deng et al., 2019)

RetinaFace, presented by Deng, Guo, Ververas, Kotsia, and Zafeiriou at CVPR 2019, is a robust single-stage face detection method that achieves state-of-the-art performance on the WIDER FACE benchmark, which includes faces across a wide range of scales, poses, occlusions, and lighting conditions. Unlike two-stage detectors that first propose candidate regions and then classify them, RetinaFace performs face detection and facial landmark prediction simultaneously in a single forward pass, substantially reducing inference latency.

A critical contribution of RetinaFace is its use of a Feature Pyramid Network (FPN) backbone, which enables the detection of faces at dramatically different scales within the same image — from large close-up faces to tiny distant faces — by fusing feature maps at multiple spatial resolutions. The network simultaneously predicts five facial landmark locations (eye centres, nose tip, and mouth corners), which can be used for precise face alignment before emotion classification, improving downstream accuracy. In EmoVision, RetinaFace is employed as the face detector in image analysis mode, where the higher computational cost is acceptable in exchange for the superior detection recall and precision it provides.

2.2.4 FER2013 Dataset: Challenges in Representation Learning (Goodfellow et al., 2013)

The FER2013 dataset was introduced by Goodfellow et al. as part of the ICML 2013 Challenges in Representation Learning workshop. The dataset contains 35,887 greyscale 48×48 pixel face images collected via automated querying of the Google Image Search API and subsequently filtered and labelled using Amazon Mechanical Turk crowdsourcing. Images are distributed across seven emotion categories: angry (4,953 images), disgust (547 images), fear (5,121 images), happy (8,989 images), sad (6,077 images), surprise (4,002 images), and neutral (6,198 images).

FER2013 has become the de facto standard benchmark for evaluating facial expression recognition systems, largely because of its size, diversity, and challenging real-world characteristics: images include significant pose variation, varying illumination, occlusion, and some degree of label noise arising from the crowdsourcing annotation process. Human performance on FER2013 has been measured at approximately 65–68%, a figure that contextualises the 66–68% accuracy achieved by EmoVision's custom model and the broader challenge of the dataset. The severe class imbalance — disgust comprises only 1.5% of the dataset, while happy represents 25% — necessitates special handling during training, addressed in EmoVision through weighted random sampling and class-balanced loss functions.

2.2.5 RAF-DB: Real-World Affective Faces Database (Li et al., 2017)

The Real-World Affective Faces Database (RAF-DB), introduced by Li, Deng, and Du and presented at CVPR 2017, addresses several limitations of FER2013. RAF-DB contains 29,672 facial images collected from the internet and annotated by 315 trained human annotators using a two-stage crowdsourcing process, resulting in

significantly higher label quality than FER2013's automated Mechanical Turk labels. The images span a broader range of subjects in terms of age, ethnicity, and face pose, and include higher-resolution imagery aligned using facial landmarks.

RAF-DB achieves human annotation agreement rates of approximately 91%, compared to estimates of 65–70% for FER2013, reflecting the substantially higher confidence in its labels. For EmoVision's custom training pipeline, RAF-DB is offered as an optional supplement to FER2013 training, using a label remapping from RAF-DB's annotation schema to the FER2013 seven-category framework. Experiments indicate that combined FER2013 and RAF-DB training can improve validation accuracy by 2–4 percentage points over FER2013 alone, suggesting that the diversity and quality improvements in RAF-DB are genuinely complementary.

2.2.6 Elastic Weight Consolidation for Continual Learning (Kirkpatrick et al., 2017)

The seminal paper on Elastic Weight Consolidation (EWC) by Kirkpatrick, Pascanu, Rabinowitz, Veness, Desjardins, Rusu, Milan, Quan, Ramalho, Grabska-Barwinska, Hassabis, Clopath, Kumaran, and Hadsell (2017), published in Proceedings of the National Academy of Sciences, addresses the fundamental problem of catastrophic forgetting in artificial neural networks — the tendency of a network to abruptly and severely lose previously acquired knowledge upon learning new tasks.

The EWC approach is motivated by an analogy to synaptic consolidation in biological neural networks, where experience-dependent changes in synaptic strength are selectively stabilised to preserve important memories. Mathematically, EWC computes the Fisher Information Matrix (FIM) diagonal — an approximation of the curvature of the loss landscape with respect to each network parameter — after training on the original task. Parameters that are highly influential for the original task have high Fisher information values; during subsequent training on new tasks, EWC adds a quadratic penalty to the loss function that resists changes to these high-importance parameters while allowing low-importance parameters to adapt freely.

In EmoVision, EWC is implemented as the core regularisation mechanism in the continual learning pipeline (continual_train.py). The Fisher Information is computed over a representative sample of 2,000 FER2013 training images after initial model training, and this information is used to protect the model's general emotion recognition capability while allowing it to adapt to the specific characteristics of Indian face images collected through the auto-data-collection feature. This approach enables the iterative improvement of model accuracy on underrepresented demographics without sacrificing the generalisation capability acquired during initial training — a practically significant achievement with clear implications for responsible AI deployment.

2.2.7 ONNX Runtime for Cross-Platform Model Deployment

The Open Neural Network Exchange (ONNX) format and its associated inference runtime (ONNX Runtime, or ORT) represent a significant practical contribution to the field of ML model deployment. ONNX was

developed collaboratively by Microsoft and Facebook (now Meta) as an open standard for representing machine learning models in a hardware-agnostic, framework-independent format. A model trained in PyTorch can be exported to ONNX format and subsequently executed using ONNX Runtime on any supported platform — CPU, GPU, or specialised accelerators — without modification.

ONNX Runtime applies a series of graph-level optimisations to the exported model before execution: operator fusion (combining adjacent operations into single efficient kernels), constant folding (pre-computing operations with static inputs), and memory planning (minimising allocation overhead). These optimisations collectively reduce CPU inference latency by 5–10x compared to native PyTorch inference, a critical improvement for EmoVision's real-time streaming use case. The custom EfficientNet-B0 model trained within EmoVision is exported to ONNX format using torch.onnx.export(), and loaded into the production inference pipeline via onnxruntime.InferenceSession(), enabling the ensemble model to contribute to predictions at negligible additional computational cost.

2.2.8 FastAPI: High-Performance Python Web Framework

FastAPI, developed by Sebastián Ramírez and released in 2018, is a modern, high-performance Python web framework built on the Starlette ASGI toolkit and leveraging Python's type annotation system for automatic request validation, serialisation, and API documentation generation. FastAPI is fully asynchronous, natively supporting both REST endpoints and WebSocket connections within a single application, and benchmarks consistently among the fastest Python web frameworks available — on par with Node.js and Go for typical API workloads.

The choice of FastAPI for EmoVision's backend was motivated by several factors: its native WebSocket support enables the real-time camera streaming architecture without requiring a separate WebSocket server or middleware layer; its asynchronous request handling allows the server to process WebSocket frames from the camera without blocking concurrent REST API requests; and its automatic OpenAPI documentation generation provides a built-in API specification that facilitates testing and integration. The Pydantic-based validation automatically rejects malformed requests before they reach the ML inference pipeline, providing an important layer of robustness.

2.2.9 WebSocket Protocol for Real-Time Communication

The WebSocket protocol (RFC 6455), standardised by the Internet Engineering Task Force (IETF) in 2011, defines a full-duplex communication channel over a single, long-lived TCP connection. Unlike the request-response model of HTTP — where each data exchange requires a complete handshake cycle — WebSocket maintains a persistent connection that allows either party (client or server) to initiate data transmission at any time without the overhead of repeated connection establishment.

For EmoVision's real-time camera streaming use case, this architectural characteristic is essential. The browser must transmit approximately 8 video frames per second to the server for analysis, and receive emotion classification results for each frame. Using HTTP for this purpose would require 8 complete request-response cycles per second per user, each with the overhead of HTTP headers, connection establishment, and TCP slow-start. WebSocket reduces this to a single connection per user, with binary or text frames transmitted bidirectionally at whatever rate the application requires. The practical result is significantly lower latency and higher throughput — enabling the real-time inference experience that is central to EmoVision's live camera feature.

2.2.10 Transfer Learning in Deep Neural Networks (Pratt, 1993; Pan and Yang, 2010)

Transfer learning — the practice of initialising a model with weights pre-trained on one task or dataset before fine-tuning on a target task — has become the dominant paradigm for practical deep learning applications. The theoretical foundation was established by Pratt (1993) and comprehensively surveyed by Pan and Yang (2010) in their influential ACM survey paper. The central insight is that deep neural networks learn hierarchical feature representations: early layers capture universal low-level features (edges, textures, colour gradients) while later layers capture increasingly task-specific high-level features. When the source task (e.g., ImageNet classification) shares structural similarity with the target task (e.g., emotion classification from faces), transferring the early-layer representations dramatically reduces the amount of labelled target-domain data required to achieve competitive accuracy.

In EmoVision's training pipeline, EfficientNet-B0 is initialised with ImageNet-1K pre-trained weights. The early feature extraction blocks (approximately the first five of eight blocks) are frozen during training, preserving the universal feature representations. Only the later blocks and the classification head are fine-tuned on FER2013, which has two important consequences: training is significantly faster (fewer parameters require gradient computation), and the risk of overfitting on FER2013's 28,000 training images is substantially reduced. This approach achieves the target accuracy range of 66–68% with 40 training epochs, compared to the 150+ epochs that would typically be required for training from random initialisation.

2.3 Comparison of Existing Emotion Recognition Systems

A systematic comparison of existing emotion recognition systems reveals a clear landscape of trade-offs along the dimensions of accuracy, speed, privacy, accessibility, and deployment complexity. The following analysis positions EmoVision relative to its principal alternatives.

Table 2.1: Comparative Analysis of Existing Emotion Recognition Systems

System / Approach	Accuracy	Real-time?	Privacy	Cost	Multi-face
Amazon Rekognition	~85–90%	Yes (cloud)	Poor (cloud)	Paid API	Yes
Microsoft Azure Face API	~83–88%	Yes (cloud)	Poor (cloud)	Paid API	Yes
Google Vision AI	~80–85%	Yes (cloud)	Poor (cloud)	Paid API	Yes
OpenCV Haar + SVM	~55–62%	Yes (local)	Good (local)	Free	Limited
FER Library (Mini-Xception)	~62–66%	Yes (local)	Good (local)	Free	Yes
DeepFace (standalone)	~66–71%	Partial	Good (local)	Free	Limited
AffectNet-trained models	~70–75%	No	Depends	Research	No
EmoVision (this work)	~66–68%	Yes (local)	Excellent (local)	Free/OSS	Yes

As the comparison table illustrates, commercially deployed cloud systems generally achieve higher accuracy but at significant cost to privacy, accessibility, and per-query pricing. Open-source local systems like the FER library and standalone DeepFace offer better privacy characteristics but typically lack real-time multi-face support, a production-ready user interface, or the kind of integrated training and continual learning pipeline that EmoVision provides. EmoVision occupies a deliberate position: competitive accuracy with full local privacy, genuine real-time capability, multi-face support, and a complete software ecosystem — representing the most practically comprehensive open-source solution in its category.

2.4 Dataset Landscape and Training Data Considerations

The availability and quality of labelled training data is the single most important determinant of emotion recognition system performance. Understanding the landscape of available datasets is essential context for evaluating EmoVision's design decisions.

FER2013 remains the most widely used benchmark dataset despite its known limitations: label noise estimated at 15–25% in some emotion categories, severe class imbalance (disgust is represented at less than 2% of the dataset), limited spatial resolution (48×48 pixels), and demographic bias toward Western populations. Its continued use as a benchmark is primarily a function of its availability and the extensive literature that has established baseline performance figures against it, enabling meaningful system comparisons.

AffectNet, containing over 450,000 images scraped from the internet and annotated by 12 trained annotators, represents the largest publicly available facial expression dataset and has yielded the highest reported accuracy

figures — models trained on AffectNet have achieved 62–65% on seven-class emotion recognition in unconstrained settings. However, AffectNet is available only for academic research use under a restrictive license, and its size makes it computationally expensive to work with without significant GPU resources. EmoVision's training pipeline supports AffectNet integration but does not require it for baseline operation.

JAFFE (Japanese Female Facial Expression) is a small but high-quality dataset of 213 images from 10 Japanese female subjects, annotated by 60 Japanese observers. While too small for training modern deep networks, it serves as a useful supplementary validation set for assessing cross-cultural generalisation. ExpW contains 91,793 images with crowd-sourced emotion labels, providing significant diversity in face pose, age, and background but with corresponding label quality trade-offs.

The demographic gap in existing datasets has direct practical consequences. Studies by Buolamwini and Gebru (2018), subsequently replicated by multiple research groups, demonstrated that facial analysis systems trained on Western-dominated datasets perform measurably worse on darker-skinned individuals, particularly women. For a system deployed in India — home to over 1.4 billion people with skin tones ranging from very light to very dark — this bias represents both a technical accuracy problem and an ethical responsibility. EmoVision's auto-data-collection feature and continual learning pipeline are directly motivated by the need to address this gap in a practical, incremental manner.

2.5 Gaps Identified in Existing Literature and Systems

The literature survey reveals several important gaps that informed EmoVision's design decisions. These gaps are documented here to provide context for the architectural and engineering choices described in subsequent chapters.

First, the overwhelming majority of published emotion recognition systems are research prototypes rather than deployable applications. They typically demonstrate accuracy on benchmark datasets but provide no user interface, no deployment infrastructure, no API, and no documentation beyond the training script. The gap between research accuracy and deployed utility is rarely addressed.

Second, continual learning for emotion recognition is underexplored in the applied literature. While EWC and related techniques are well-established in the theoretical continual learning literature, their practical application to building emotion recognition systems that improve continuously in deployment — rather than requiring periodic full retraining — has received limited attention.

Third, demographic bias correction in emotion recognition systems is typically treated as a dataset-level problem (collect more diverse data) rather than an architecture-level problem (build systems that can adapt to new demographics in deployment). EmoVision's approach — combining an auto-collection feedback loop with EWC-protected continual fine-tuning — represents a practical architectural response to this challenge.

Fourth, the real-time multi-face tracking problem — maintaining consistent identity assignments for multiple faces across video frames while simultaneously running emotion classification — is rarely addressed holistically in the literature. EmoVision's IoU-based face tracking with per-face temporal smoothing buffers addresses this directly.

These identified gaps collectively define the contribution space that EmoVision occupies: a production-deployable, privacy-preserving, continually learning, multi-face real-time emotion recognition system with an integrated user interface and deployment infrastructure.

CHAPTER 3: SPECIFICATIONS OVERVIEW & SYSTEM DESIGN

3.1 Hardware Requirements

The hardware requirements for EmoVision are specified at two tiers — minimum and recommended — to accommodate the broadest range of deployment contexts while clearly communicating the performance implications of hardware choice.

Table 3.1: Hardware Requirements Specification

Component	Minimum Specification	Recommended Specification	Performance Impact
Processor (CPU)	Intel Core i5 (4-core, 2.0 GHz) / AMD Ryzen 5	Intel Core i7/i9 (8-core+) / AMD Ryzen 7/9	More cores accelerate OpenCV parallel ops; base clock determines per-frame latency
System RAM	8 GB DDR4	16 GB+ DDR4	TF + DeepFace weights require ~1.5–2 GB at runtime; 8 GB allows comfortable operation
Graphics (GPU)	Not required	NVIDIA GTX 1060 6 GB+ (CUDA 11.8+)	GPU accelerates live inference from 8–10 FPS to 25–60 FPS
Storage	5 GB free (SSD/HDD)	20 GB+ free SSD	SSD dramatically reduces model weight load time on startup; TF install is ~3 GB
Webcam	720p USB webcam	1080p 30 FPS USB/built-in	Higher resolution improves detection at distance; 720p fully sufficient for ≤2m range
Network	Localhost only (dev)	Port 8000 accessible	Required only for initial model weight download (~600 MB on first launch)
Operating System	Windows 10 / Ubuntu 20.04 / macOS 12	Windows 11 / Ubuntu 22.04 LTS	All platforms supported; Ubuntu recommended for Docker deployment
Python Runtime	Python 3.10	Python 3.12	3.12 offers 10–15% performance improvement over 3.10 on I/O-bound workloads
Web Browser	Chrome 90+ / Firefox 88+ / Edge 90+	Latest Chrome / Edge	WebRTC camera access and WebSocket are required; Safari has partial WebRTC support

Table 3.2: Expected Performance by Hardware Configuration

Hardware Configuration	Image Analysis (per image)	Live Stream FPS	Recommended Use
Entry CPU (i5 / Ryzen 5, 4-core)	2–5 seconds	3–5 FPS	Development, occasional use
Mid CPU (i7 / Ryzen 7, 8-core)	0.8–2 seconds	6–10 FPS	Demos, academic research, classroom
High CPU (i9 / Ryzen 9, 12-core+)	0.4–1 second	10–15 FPS	Professional deployment on CPU
NVIDIA GTX 1060 6 GB (CUDA)	0.1–0.3 seconds	25–40 FPS	Production deployment with smooth real-time
NVIDIA RTX 3060+ (CUDA)	< 0.1 seconds	50–60+ FPS	High-throughput production; multi-user

3.2 Software Requirements

EmoVision's software requirements span the full stack from operating system and runtime through to frontend dependencies. All software components are open-source unless otherwise noted.

Table 3.3: Software Requirements Specification

Category	Package / Tool	Version	Purpose
Runtime	Python	3.10–3.12	Core application language
Web Framework	FastAPI	≥ 0.110.0	REST API and WebSocket server
Web Framework	Uvicorn[standard]	≥ 0.27.0	ASGI server for async handling
Web Framework	python-multipart	≥ 0.0.9	Multipart file upload parsing
ML — Primary	DeepFace	≥ 0.0.90	Primary emotion detection engine
ML — Fallback	FER	≥ 22.5.1	Fallback emotion backend (MTCNN)
ML — Bridge	tf-keras	≥ 2.18.0	Keras bridge for TensorFlow 2.20+
ML — Framework	TensorFlow	≥ 2.15.0	DeepFace's underlying DL framework
ML — Framework	PyTorch	≥ 2.2.0	Custom EfficientNet-B0 training

Category	Package / Tool	Version	Purpose
ML — Framework	torchvision	≥ 0.17.0	Pre-trained models and transforms
ML — Export	ONNX	≥ 1.15.0	Cross-platform model export format
ML — Export	onnxruntime	≥ 1.17.0	Fast CPU inference from ONNX models
Computer Vision	OpenCV-headless	≥ 4.9.0	Image decoding, annotation, ops
Computer Vision	Pillow	≥ 10.2.0	Image I/O and format conversion
Numerics	NumPy	≥ 1.26.0	Array operations throughout pipeline
Data	Pandas	≥ 2.2.0	FER2013 CSV dataset loading
Evaluation	scikit-learn	≥ 1.4.0	Confusion matrix, train/val split
Visualisation	matplotlib / seaborn	≥ 3.8.0	Training curves, confusion heatmaps
Deployment	Docker	≥ 24.0	Container-based deployment
Frontend	Chart.js	4.4.0 (CDN)	Session analytics visualisation
Frontend	Google Fonts	CDN	Syne, IBM Plex Mono typography
Version Control	Git	≥ 2.40	Source control and deployment CI

3.3 System Architecture

EmoVision follows a three-tier architecture pattern that cleanly separates presentation, business logic, and data concerns. This separation enables independent development and testing of each tier, simplifies deployment, and allows each component to be scaled or replaced independently as requirements evolve.

3.3.1 Architecture Overview

The three tiers of the EmoVision architecture are defined as follows. The Presentation Tier is the browser-based Single Page Application, implemented in vanilla HTML5, CSS3, and JavaScript with no build step requirement. The Business Logic Tier is the FastAPI Python server, which handles request routing, WebSocket session management, input validation, and orchestration of the ML inference pipeline. The Data Tier encompasses the ML models (DeepFace weights, custom ONNX model), training datasets, and the auto-collected data store.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        PRESENTATION TIER (Browser)                       │
│  ┌──────────────────────────────┐   ┌──────────────────────────────────┐  │
│  │    Image Upload Mode         │   │      Live Camera Mode            │  │
│  │  Drag-drop → Preview →       │   │  getUserMedia() → Canvas Capture │  │
│  │  Analyze Button → Results    │   │  → WebSocket Send → Overlay      │  │
│  └──────────────┬───────────────┘   └──────────────┬───────────────────┘  │
│                 │  POST /api/analyze                │  WS /ws/stream       │
└─────────────────┼───────────────────────────────────┼──────────────────────┘
                  ▼                                   ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                     BUSINESS LOGIC TIER (FastAPI Server)                 │
│  ┌──────────────────┐   ┌──────────────────┐   ┌────────────────────────┐ │
│  │ REST API Handler │   │ WebSocket Handler│   │ Health / Static Serving │ │
│  │ POST /api/analyze │   │ /ws/stream       │   │ GET /api/health  GET /  │ │
│  └────────┬─────────┘   └────────┬─────────┘   └────────────────────────┘ │
│           │                      │                                        │
│           └──────────┬───────────┘                                        │
│                      ▼                                                    │
│              ┌──────────────────────────────────────────┐                │
│              │        EmotionPipeline (pipeline.py)     │                │
│              │  DeepFace+RetinaFace → ONNX Ensemble →   │                │
│              │  Temporal Smoother → Confidence Gate →   │                │
│              │  draw_annotations() → JSON Response      │                │
│              └──────────────────────────────────────────┘                │
└──────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                              DATA TIER                                   │
│  ┌────────────────┐  ┌────────────────────┐  ┌──────────────────────────┐ │
│  │ DeepFace Models│  │ finetuned_model    │  │ collected_data/ (auto-save)│ │
│  │ (~600 MB auto  │  │ .onnx (custom      │  │ FER2013 CSV (training)   │ │
│  │  download)     │  │  trained)          │  │ fisher.pt (EWC)          │ │
│  └────────────────┘  └────────────────────┘  └──────────────────────────┘ │
```

└──┘

Figure 3.1: EmoVision Three-Tier System Architecture Diagram

3.3.2 Request Lifecycle — Image Analysis Mode

When a user uploads an image through the web interface, the following sequence of operations occurs. Understanding this lifecycle is essential for appreciating both the performance characteristics and the architectural design decisions of the system.

11. The user selects or drags a JPEG/PNG/WebP image file into the browser's drop zone. The JavaScript FileReader API reads the file contents and renders a preview in the interface without any server communication.
12. Upon clicking the Analyze button, the JavaScript frontend constructs a multipart/form-data HTTP POST request containing the image file and sends it to the FastAPI endpoint at POST /api/analyze.
13. FastAPI's python-multipart middleware receives and parses the upload, making the file contents available as a bytes object to the endpoint handler function.
14. The endpoint decodes the bytes stream using NumPy's frombuffer() and OpenCV's imdecode() functions, converting the raw bytes into an OpenCV BGR image array of dimensions H×W×3.
15. The image array is passed to EmotionPipeline.analyze(fast_mode=False). The pipeline calls DeepFace.analyze() with the RetinaFace detector backend, which performs face detection and emotion classification in a single forward pass.
16. If a custom ONNX model is available (models/finetuned_model.onnx), the ensemble method crops each detected face, pre-processes it to 112×112 pixels, and runs it through the ONNX model. The predictions from DeepFace and ONNX are blended with a configurable weight (default 55% DeepFace, 45% ONNX).
17. The final emotion predictions (now blended and confidence-gated) are passed to draw_annotations(), which renders corner brackets, emotion probability bars, confidence labels, and face ID badges onto a copy of the original image using OpenCV drawing primitives.
18. The annotated image is JPEG-encoded and Base64-encoded, then returned in the JSON response alongside the face data array containing bounding boxes, emotion dictionaries, and confidence scores.
19. The frontend receives the JSON response, updates the result image element with the annotated Base64 image, and populates the analytics panel with emotion bars, confidence percentages, and inference time.

3.3.3 Request Lifecycle — Live Camera Mode

The live camera mode follows a different, WebSocket-based lifecycle that enables real-time streaming without the overhead of repeated HTTP connections.

20. The user clicks Start Camera. The browser requests camera access via navigator.mediaDevices.getUserMedia() and, on user permission, begins rendering the camera stream to a <video> element with a CSS horizontal mirror transform applied.
21. A JavaScript setInterval() loop fires every 120 milliseconds (approximately 8 frames per second). Each iteration draws the current video frame to a hidden <canvas> element using drawImage(), then exports the canvas contents as a JPEG-encoded Base64 string via toDataURL('image/jpeg', 0.7).
22. The Base64 frame string is transmitted through the open WebSocket connection to the FastAPI /ws/stream endpoint.
23. The server-side WebSocket handler decodes the Base64 string, creates an OpenCV image array, and calls EmotionPipeline.analyze(fast_mode=True). In fast mode, the OpenCV DNN detector is used instead of RetinaFace, significantly reducing per-frame latency.
24. The temporal smoothing FaceSmoother updates the rolling emotion buffer for each tracked face, using IoU overlap to match face detections across consecutive frames. The smoothed predictions are confidence-gated, and the final results are packaged into a JSON response.
25. The JSON response (faces array, FPS estimate, inference latency) is sent back through the WebSocket. The frontend receives it and calls drawLiveOverlay(), which renders the AR overlay directly onto the <canvas> element layered over the video. Critically, the canvas does not use CSS mirroring; instead, face bounding box X-coordinates are programmatically mirrored (x_canvas = frame_width - x_face - face_width) to align correctly with the CSS-mirrored video.

3.4 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates the movement of data between the system's external entities, processes, and data stores, providing a high-level view of information flow independent of implementation technology.

3.4.1 Level 0 (Context Diagram)

```
                User/Browser
                     |
        Image Upload  | Camera Frames (WS)
             ────────▼────────
            |                 |
            |    EmoVision    |──────► Emotion Results
            |     System      |        Annotated Images
            |                 |        Analytics Data
             ────────────────
                     |
            Model Weights | Training Data
                     |
            External Datasets (FER2013 / RAF-DB)
```

Figure 3.2: Level 0 Context DFD — EmoVision System

3.4.2 Level 1 DFD

```
User ──► [1.0 Input Handler] ──► Image Array ──► [2.0 Face Detector]
                |                                        |
     Camera     |                              Face Bounding Boxes
     Frames     |                                        |
                |                              [3.0 Emotion Classifier]
                |                                        |
                |                              Raw Emotion Probabilities
                |                                        |
                |                              [4.0 Ensemble Merger]
                |                                   |         |
                |                            DeepFace|   ONNX Model
                |                              Pred  |   Predictions
                |                                   ▼
                |                           Blended Probabilities
```

```
                |                                   |
                |                          [5.0 Temporal Smoother]
                |                                   |
                |                          Smoothed Predictions
                |                                   |
                |                          [6.0 Confidence Gate]
                |                                   |
                └──────────────────────► [7.0 Annotation Renderer]
                                                    |
                                        JSON Response + Annotated Image
                                                    |
                                              ◄ User/Browser
```

Figure 3.3: Level 1 DFD — EmoVision Processing Pipeline

3.5 UML Diagrams

3.5.1 Use Case Diagram

The Use Case Diagram captures the interactions between the system's actors and its functional capabilities. EmoVision has two primary actors: the End User who interacts with the web interface, and the System Administrator who manages training and deployment.

```
    ┌──────────────────────────────────────────────────────────────────┐
    |                    EmoVision System                              |
    |                                                                  |
    |     (UC1) Upload and Analyze Image                               |
    |     (UC2) Activate Live Camera Feed                              |
    |     (UC3) View Emotion Analytics Dashboard                       |
    |     (UC4) Select Individual Face in Multi-face Scenario          |
    |     (UC5) Take Camera Snapshot                                   |
    |     (UC6) Change/Reset Uploaded Image                            |
    |     (UC7) Adjust Camera Feed Size                                |
```

```
    |     (UC8) Train Custom Model (train.py)                          |
    |     (UC9) Fine-tune with Continual Learning                      |
    |     (UC10) Enable Auto Data Collection                           |
    |     (UC11) Deploy System via Docker                              |
    └──────────────────────────────────────────────────────────────────┘

          ▲                                ▲

    ┌──────┴──────┐                  ┌──────┴──────────────┐
    |  End User   |                  |  System Administrator |
    | (UC1-UC7)   |                  | (UC8-UC11)            |
    └─────────────┘                  └──────────────────────┘
```

Figure 3.4: Use Case Diagram — EmoVision System Actors and Functions

3.5.2 Class Diagram

The Class Diagram captures the principal classes in the EmoVision system, their attributes, methods, and relationships.

```
    ┌──────────────────────────────────┐     ┌──────────────────────────────┐
    |         EmotionPipeline          |     |        FaceSmoother          |
    |──────────────────────────────────|     |──────────────────────────────|
    | - backend: str                   |     | - window: int                |
    | - onnx_model: OnnxEmotionModel   |     | - buffers: list[deque]       |
    | - smoother: FaceSmoother         |     | - boxes: list[dict]          |
    | - _frame_no: int                 |     |──────────────────────────────|
    |──────────────────────────────────|     | + update(results): list      |
    | + analyze(img, fast_mode): list  |◄────| + reset()                    |
    |+ draw_annotations(img,res):ndarray|    | - _iou(a,b): float           |
    | + model_info(): dict             |     | - _average(buf): dict        |
    | - _run_deepface(img,fast): list  |     └──────────────────────────────┘
    | - _ensemble(img,results): list   |
    | - _apply_gate(results): list     |     ┌──────────────────────────────┐
    | - _maybe_collect(img,res): void  |     |     OnnxEmotionModel         |
```

```
└──────────────────────────────────┘      │──────────────────────────────│
                                          │ - session: ort.Session       │
┌──────────────────────────────────┐      │ - input_name: str            │
│    FastAPI Application (main)    │      │ - input_h, input_w: int      │
│──────────────────────────────────│      │──────────────────────────────│
│ - pipeline: EmotionPipeline      │      │ + predict(face_bgr): dict    │
│──────────────────────────────────│      └──────────────────────────────┘
│ + serve_frontend(): HTMLResponse │
│ + analyze_image(file): JSONResp  │   ┌──────────────────────────────┐
│ + websocket_stream(ws): void     │   │ EfficientNetTrainer(train.py)│
│ + health(): dict                 │   │──────────────────────────────│
└──────────────────────────────────┘   │ + train(args): void          │
                                       │ + build_model(): nn.Module   │
                                       │ + get_class_weights(): Tensor│
                                       └──────────────────────────────┘
```

Figure 3.5: Class Diagram — EmoVision Core Components

3.5.3 Sequence Diagram — Image Analysis

```
User        Browser JS        FastAPI           EmotionPipeline    DeepFace
 │              │               │                    │               │
 │ Upload Image │               │                    │               │
 │──────────────►│               │                    │               │
 │              │ POST /analyze │                    │               │
 │              │──────────────►│                    │               │
 │              │               │ analyze(img,False) │               │
 │              │               │────────────────────►│               │
 │              │               │                    │ analyze(img)  │
 │              │               │                    │──────────────►│
 │              │               │                    │ face + emotion│
 │              │               │                    │◄──────────────│
 │              │               │                    │ onnx.predict()│
 │              │               │                    │ (if available)│
```

```
|              |              |                     | blend+smooth  |
|              |              |                     | confidence_gate|
|              |              |                     | draw_annots() |
|              |              | faces+image(b64)    |               |
|              |              |◄────────────────────|               |
|              | JSON response |                    |               |
|              |◄─────────────|                     |               |
| Show results |              |                     |               |
|◄─────────────|              |                     |               |
```

Figure 3.6: Sequence Diagram — Image Analysis Request

3.5.4 Sequence Diagram — Live Camera Streaming

```
User   Browser JS     WebSocket       FastAPI WS        EmotionPipeline
 |          |            |  (persistent)    |                 |
 | Start Cam|            |                  |                 |
 |──────────►| getUserMedia|                |                 |
 |          | WS connect |                  |                 |
 |          |────────────►| accept()        |                 |
 |          |            |─────────────────►|                 |
 |          |            |                  |                 |
 |  [every 120ms loop]   |                  |                 |
 |          | base64 frame|                 |                 |
 |          |────────────►|─────────────────►|                 |
 |          |            |                  |analyze(img,True)|
 |          |            |                  |─────────────────►|
 |          |            |                  | smoothed results|
 |          |            |                  |◄────────────────|
 |          |            | JSON{faces,fps}  |                 |
 |          |◄───────────|◄─────────────────|                 |
 |          | drawOverlay|                  |                 |
 | See overlay           |                  |                 |
 |◄─────────|            |                  |                 |
```

```
│          │          [loop continues until stop]          │
```

Figure 3.7: Sequence Diagram — Live Camera WebSocket Streaming

3.5.5 Activity Diagram — System Processing Flow

```
                    ┌─────────┐
                    │ START   │
                    └────┬────┘
                         │
                ┌────────▼────────┐
                │User selects mode │
                └────────┬────────┘
          ┌──────────────┼──────────────┐
  Image Mode             │             Camera Mode
          │              │                 │
   ┌──────▼──────┐       │      ┌──────────▼──────────┐
   │Upload Image File│   │      │ Request Camera Permission│
   └──────┬──────┘       │      └──────────┬──────────┘
          │              │                 │
   ┌──────▼──────┐       │      ┌──────────▼──────────┐
   │POST /api/analyze│   │      │Open WebSocket Connection │
   └──────┬──────┘       │      └──────────┬──────────┘
          │              │                 │ [Frame Loop]
   ┌──────▼──────┐       │      ┌──────────▼──────────┐
   │ Decode → Detect │   │      │ Capture → Base64 → Send  │
   │ Classify → Blend│   │      │ Receive → Smooth → Gat   │
   │ Gate → Annotate │   │      │ Overlay → Display        │
   └──────┬──────┘       │      └──────────┬──────────┘
          │              │                 │ [User stops]
   ┌──────▼──────┐       │      ┌──────────▼──────────┐
   │ Return JSON +   │   │      │ Close WebSocket          │
   │ Annotated Image │   │      │ Stop camera stream       │
   └──────┬──────┘       │      └──────────┬──────────┘
          └──────────────┼──────────────┘
```

```
                    ┌──────────▼──────────┐
                    │       END           │
                    └─────────────────────┘
```

Figure 3.8: Activity Diagram — EmoVision User Interaction Flow

3.6 ML Pipeline Design

The Machine Learning inference pipeline is the technical core of EmoVision. Its design reflects a series of deliberate trade-offs between accuracy, latency, robustness, and maintainability. This section documents those design decisions and their rationale.

3.6.1 Dual Backend Architecture

The inference engine is designed around a primary-fallback backend pattern. DeepFace with the RetinaFace detector is the primary backend, selected for its superior face detection recall in challenging conditions (extreme pose, low light, occlusion) and its strong baseline emotion classification accuracy. If DeepFace fails to install or initialise (e.g., in restricted environments), the FER library with MTCNN detection provides an automatic fallback that maintains the system's functional capability at slightly reduced accuracy.

This architectural choice ensures that EmoVision remains functional across a wide range of deployment environments without requiring manual configuration. The backend selection is performed once at startup by the EmotionPipeline initialiser, and the result is cached for the lifetime of the server process, adding no per-request overhead.

3.6.2 Ensemble Model Design

The ensemble mechanism blends the probability outputs of DeepFace and the custom ONNX model using a weighted linear combination. For a given face, the final emotion probability for class c is computed as:

$$P_final(c) = (1 - \alpha) \times P_deepface(c) + \alpha \times P_onnx(c)$$

where α is the ENSEMBLE_WEIGHT parameter (default 0.45). This is applied independently for each of the seven emotion classes, and the resulting distribution is re-normalised to sum to 1.0. The dominant emotion is then extracted as argmax(P_final).

The rationale for this weighting is that DeepFace's underlying model has been trained on substantially larger and more diverse data than the custom ONNX model, providing a strong general prior. The ONNX model contributes domain-specific adjustment — particularly relevant for the demographic correction use case —

without overriding the general knowledge. The α parameter can be adjusted at the top of pipeline.py; users who have extensively fine-tuned their ONNX model may prefer α values closer to 0.6–0.7.

3.6.3 Temporal Smoothing Design

Real-time emotion detection from video exhibits frame-to-frame prediction variance that makes raw per-frame predictions appear to 'flicker' — the dominant emotion label may switch rapidly between adjacent categories (e.g., happy → surprise → happy) even as the underlying facial expression remains stable. This visual noise is perceptually jarring and reduces the utility of the system for real-time applications.

The FaceSmoother class addresses this by maintaining a rolling deque buffer of the last N emotion probability distributions for each tracked face, and returning the mean distribution across all buffered frames. The default window N = 5 frames represents approximately 600 milliseconds of smoothing at 8 FPS — long enough to eliminate rapid flickering while short enough to respond to genuine emotion changes within one second. This window is configurable via the SMOOTH_WINDOW constant in pipeline.py.

3.6.4 Confidence Gate Design

The confidence gate is a post-processing filter that prevents the system from displaying emotion labels when its predictions are too uncertain to be meaningful. When the probability of the top-ranked emotion falls below the CONFIDENCE_GATE threshold (default 0.40, or 40%), the system overrides the dominant emotion display with 'neutral' and suppresses the prediction confidence value.

This design decision reflects the observation that uncertain model predictions — those where probability mass is spread relatively evenly across multiple emotion categories — are more likely to reflect genuine face detection or expression ambiguity than a meaningful emotional signal. Displaying a spurious label in such cases would mislead users. The 'neutral' fallback is chosen because it represents the absence of a strong expressed emotion, which is the most honest representation of an uncertain prediction. The threshold value was calibrated empirically to balance responsiveness with false positive suppression.

Table 3.4: ML Pipeline Design Parameters and Their Values

Parameter	Default Value	Description	Adjustment Guidance
SMOOTH_WINDOW	5 frames	Rolling average window for temporal smoothing	Increase for smoother display; decrease for faster response to expression changes
CONFIDENCE_GATE	0.40 (40%)	Minimum confidence to show non-neutral label	Increase for stricter filtering; set to 0 to disable and show all predictions
ENSEMBLE_WEIGHT	0.45	Weight of custom ONNX model in ensemble blend	Increase toward 1.0 as confidence in fine-tuned model grows

Parameter	Default Value	Description	Adjustment Guidance
COLLECT_DATA	False	Enable auto-collection of high-confidence crops	Set True during extended live sessions to build training data
COLLECT_THRESHOLD	0.75	Confidence threshold for auto-collected samples	Higher value → fewer but higher-quality collected training samples
COLLECT_DIR	collected_data	Root directory for auto-collected images	Can be set to a networked share for multi-device collection

3.7 Database and Data Store Design

EmoVision does not use a traditional relational database for operational data; all inference results are computed on-demand and returned to the client as JSON without persistent storage. This design choice is intentional: it ensures zero persistent user data on the server, aligning with the system's privacy-first philosophy.

The data stores used by the system are all file-system based and structured as follows.

Model Weights Store (models/): Contains the custom EfficientNet-B0 checkpoint (best_model.pth, finetuned_model.pth), the ONNX export (finetuned_model.onnx), and the EWC Fisher Information Matrix (fisher.pt). These files are typically 50–200 MB in total and are excluded from version control via .gitignore.

Auto-Collected Data Store (collected_data/): Organised as a directory tree with one subdirectory per emotion class. JPEG face crop images are named by Unix timestamp in milliseconds, ensuring uniqueness without requiring a database index. This directory grows organically during live operation and is excluded from version control.

Training Datasets (data/): Contains the FER2013 CSV file and optionally the RAF-DB image tree. These are large binary data that are downloaded separately via the Kaggle API and excluded from version control.

Static Frontend Assets (static/): Contains the single index.html file that constitutes the entire frontend application. This file is served by FastAPI as a static file on GET / requests, requiring no build step, bundler, or preprocessing.

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment Setup

The EmoVision system was developed on a Windows 11 machine equipped with an Intel Core i7 processor, 16 GB RAM, and a 1 TB NVMe SSD. The primary development tools were Visual Studio Code with Python and REST Client extensions, Git for version control with the remote repository hosted on GitHub at ManuAnand3003/EmoVision, and Windows PowerShell for command-line operations. Python 3.12 was used with a dedicated virtual environment created via the venv module to isolate project dependencies from the system Python installation.

The development process followed the Agile sprint methodology described in Chapter 3, with each sprint producing a demonstrable, functional increment of the system. Sprint 1 established the basic FastAPI skeleton and DeepFace integration. Sprint 2 added the WebSocket streaming pipeline and live camera frontend. Sprint 3 introduced the analytics dashboard, multi-face tracking, and the polished landing-page UI. Sprint 4 completed the training pipeline, ONNX export, ensemble inference, temporal smoothing, and continual learning components.

4.2 Backend Implementation

4.2.1 FastAPI Application Structure (main.py)

The FastAPI application is the entry point for all server-side functionality. It is intentionally compact — approximately 100 lines of code — because it acts purely as an orchestrator, delegating all heavy computation to the EmotionPipeline class. This separation of concerns makes the codebase easy to maintain and test independently.

```
from fastapi import FastAPI, File, UploadFile, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from engine.pipeline import EmotionPipeline
import base64, cv2, numpy as np, time, traceback
from pathlib import Path

app = FastAPI(title="EmoVision API", version="2.0.0")

app.add_middleware(CORSMiddleware, allow_origins=["*"],
```

```
    allow_methods=["*"], allow_headers=["*"])


# Mount static files and load pipeline once at startup
app.mount("/static", StaticFiles(directory="static"), name="static")
pipeline = EmotionPipeline()  # Initialises backend and loads models


@app.get("/", response_class=HTMLResponse)
async def serve_frontend():
    html_path = Path("static/index.html")
    return HTMLResponse(content=html_path.read_text(encoding="utf-8"))


@app.post("/api/analyze")
async def analyze_image(file: UploadFile = File(...)):
    try:
        contents = await file.read()
        nparr = np.frombuffer(contents, np.uint8)
        img_bgr = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
        if img_bgr is None:
            return JSONResponse(status_code=400,
                content={"error": "Could not decode image. Upload JPG/PNG."})
        start = time.perf_counter()
        results = pipeline.analyze(img_bgr)  # fast_mode=False for images
        elapsed_ms = round((time.perf_counter() - start) * 1000, 1)
        annotated = pipeline.draw_annotations(img_bgr.copy(), results)
        _, buffer = cv2.imencode(".jpg", annotated,
                                 [cv2.IMWRITE_JPEG_QUALITY, 92])
        annotated_b64 = "" + \
                        base64.b64encode(buffer).decode("utf-8")
        return {"success": True, "inference_ms": elapsed_ms,
                "face_count": len(results), "faces": results,
                "annotated_image": annotated_b64}
    except Exception as e:
        traceback.print_exc()
        return JSONResponse(status_code=500, content={"error": str(e)})
```

Figure 4.1: FastAPI Main Application — Image Analysis Endpoint (main.py)

4.2.2 WebSocket Streaming Handler

The WebSocket handler manages the persistent bidirectional communication channel for the live camera mode. Each client connection creates an independent WebSocket session that persists until the user stops the camera or closes the browser tab. A rolling deque is used to compute a smooth FPS estimate over a two-second window, which is included in every JSON response to inform the frontend's FPS display.

```
@app.websocket("/ws/stream")
async def websocket_stream(websocket: WebSocket):
    await websocket.accept()
    frame_times = []
    try:
        while True:
            data = await websocket.receive_text()
            frame_start = time.perf_counter()

            # Strip data URI prefix if present
            if data.startswith("data:image"):
                data = data.split(",", 1)[1]

            # Decode Base64 frame to OpenCV array
            img_bytes = base64.b64decode(data)
            nparr = np.frombuffer(img_bytes, np.uint8)
            frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
            if frame is None:
                continue

            # Run fast inference (OpenCV detector for speed)
            results = pipeline.analyze(frame, fast_mode=True)
            elapsed_ms = round((time.perf_counter() - frame_start) * 1000, 1)

            # Rolling FPS: count frames in last 2 seconds
            frame_times.append(time.perf_counter())
```

```
            frame_times = [t for t in frame_times
                           if time.perf_counter() - t < 2.0]
            fps = round(len(frame_times) / 2.0, 1)

            await websocket.send_json({"faces": results,
                "inference_ms": elapsed_ms, "fps": fps,
                "face_count": len(results)})

    except WebSocketDisconnect:
        print("[WS] Client disconnected.")
    except Exception as e:
        traceback.print_exc()
        await websocket.send_json({"error": str(e)})
```

Figure 4.2: WebSocket Handler — Real-Time Camera Streaming (main.py)

4.3 ML Pipeline Implementation

4.3.1 EmotionPipeline Class

The EmotionPipeline class is the system's ML inference engine. Its __init__ method performs backend detection and model loading at server startup, meaning all model weights are loaded into memory once and cached for the server's lifetime. This is critical for performance: loading TensorFlow and DeepFace models on every request would add 3–8 seconds of latency to each analysis, making real-time operation impossible.

```
class EmotionPipeline:
    def __init__(self):
        self.backend    = self._init_backend()
        self.onnx_model = self._load_onnx()   # None if not available
        self.smoother   = FaceSmoother(window=SMOOTH_WINDOW)
        self._frame_no  = 0
        if COLLECT_DATA:
            Path(COLLECT_DIR).mkdir(parents=True, exist_ok=True)
            for e in EMOTIONS:
                (Path(COLLECT_DIR) / e).mkdir(exist_ok=True)
```

```
    def _init_backend(self) -> str:
        """Attempt DeepFace, fall back to FER on ImportError."""
        try:
            from deepface import DeepFace
            dummy = np.zeros((100, 100, 3), dtype=np.uint8)
            try:  # warm-up pass to force model weight loading
                DeepFace.analyze(dummy, actions=["emotion"],
                    detector_backend="opencv", silent=True,
                    enforce_detection=False)
            except Exception:
                pass
            self._deepface = DeepFace
            return "deepface"
        except ImportError:
            pass
        try:
            from fer import FER
            self._fer = FER(mtcnn=True)
            return "fer"
        except ImportError:
            pass
        raise RuntimeError("No backend. pip install deepface OR pip install fer")
```

Figure 4.3: EmotionPipeline — Backend Initialisation and Warm-up

4.3.2 Ensemble and Numpy Type Safety

A critical implementation detail that was discovered and resolved during development was the JSON serialisation incompatibility of NumPy scalar types. DeepFace returns emotion probabilities as numpy.float32 values, which FastAPI's default JSON encoder cannot serialise, producing a TypeError at the response serialisation stage. The fix requires explicit type casting of all numpy values to native Python types before they are included in the response.

```
def _analyze_deepface(self, img_bgr, fast_mode):
```

```
    detector = "opencv" if fast_mode else "retinaface"
    results  = []
    try:
        faces = self._deepface.analyze(
            img_bgr, actions=["emotion"],
            detector_backend=detector,
            enforce_detection=False, silent=True)
        if isinstance(faces, dict): faces = [faces]

        for i, face in enumerate(faces):
            region     = face.get("region", {})
            raw_emo    = face.get("emotion", {})

            # CRITICAL: cast numpy.float32 → Python float for JSON safety
            total    = float(sum(raw_emo.values())) or 1.0
            emotions = {k.lower(): round(float(v) / total, 4)
                        for k, v in raw_emo.items()}

            dominant   = str(face.get("dominant_emotion",
                             max(emotions, key=emotions.get)))
            confidence = float(emotions.get(dominant, 0))

            results.append({
                "face_id":          i,
                "box": {            # cast region values to int
                    "x": int(region.get("x", 0)),
                    "y": int(region.get("y", 0)),
                    "w": int(region.get("w", 0)),
                    "h": int(region.get("h", 0)),},
                "emotions":         emotions,
                "dominant_emotion": dominant,
                "confidence":       round(confidence, 4),
                "color_hex":        self._face_color_hex(i),})
```

```
    except Exception as e:
        print(f"[DeepFace] Error: {e}")
    return results
```

Figure 4.4: DeepFace Analysis with NumPy Type Safety Casting

4.3.3 Temporal Smoother Implementation

The FaceSmoother class implements the IoU-based face tracking and rolling-window emotion averaging described in the design chapter. The IoU computation uses pure Python arithmetic on the bounding box dictionary values, avoiding any dependency on NumPy in this component for maximum portability.

```
class FaceSmoother:
    def __init__(self, window=5):
        self.window  = window
        self.buffers = []   # list of deques, one per tracked face
        self.boxes   = []   # last known box per tracked face

    def _iou(self, a, b):
        """Intersection over Union for two {x,y,w,h} dicts."""
        ax1,ay1 = a["x"], a["y"]
        ax2,ay2 = a["x"]+a["w"], a["y"]+a["h"]
        bx1,by1 = b["x"], b["y"]
        bx2,by2 = b["x"]+b["w"], b["y"]+b["h"]
        ix = max(0, min(ax2,bx2) - max(ax1,bx1))
        iy = max(0, min(ay2,by2) - max(ay1,by1))
        inter = ix * iy
        union = (ax2-ax1)*(ay2-ay1)+(bx2-bx1)*(by2-by1)-inter
        return inter/union if union>0 else 0.0

    def update(self, results):
        """Match results to tracked faces, update buffers, return smoothed."""
        if not results:
            return []
        smoothed = []
        for face in results:
```

```
            best_idx, best_iou = -1, 0.3
            for i, box in enumerate(self.boxes):
                iou = self._iou(face["box"], box)
                if iou > best_iou:
                    best_iou, best_idx = iou, i
            if best_idx == -1:
                self.buffers.append(deque(maxlen=self.window))
                self.boxes.append(face["box"])
                best_idx = len(self.buffers) - 1
            else:
                self.boxes[best_idx] = face["box"]
            self.buffers[best_idx].append(face["emotions"])
            face["emotions"] = self._average(self.buffers[best_idx])
            face["dominant_emotion"] = max(face["emotions"],
                                           key=face["emotions"].get)
            face["confidence"] = round(face["emotions"]
                                       [face["dominant_emotion"]], 4)
            smoothed.append(face)
        return smoothed
```

Figure 4.5: FaceSmoother — IoU Tracking and Rolling Average Implementation

4.4 Custom Model Training Implementation

4.4.1 EfficientNet-B0 Architecture and Fine-Tuning Strategy

The custom training pipeline trains EfficientNet-B0 through a two-phase approach. In the first phase, the model is initialised with ImageNet pre-trained weights and all early feature extraction blocks (layers 0–4 of the features Sequential) are frozen. Only the final three feature blocks and the custom classification head are trainable, reducing the effective parameter count to approximately 2.1 million trainable parameters out of the total 5.3 million. This preserves the general feature representations learned from ImageNet while allowing the task-specific layers to adapt to the emotion recognition domain.

```
def build_model(num_classes=7, pretrained=True):
    weights = EfficientNet_B0_Weights.IMAGENET1K_V1 if pretrained else None
```

```
    model = models.efficientnet_b0(weights=weights)

    # Freeze all parameters first
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze last 3 feature extraction blocks
    for block in list(model.features.children())[-3:]:
        for param in block.parameters():
            param.requires_grad = True

    # Replace classification head
    in_features = model.classifier[1].in_features   # 1280
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.4, inplace=True),
        nn.Linear(in_features, 512),
        nn.ReLU(),
        nn.Dropout(p=0.3),
        nn.Linear(512, num_classes),   # 7 emotion classes
    )
    return model
```

Figure 4.6: EfficientNet-B0 Architecture with Custom Classification Head

4.4.2 Class Imbalance Handling

The FER2013 dataset has severe class imbalance: the happy class contains 8,989 images while disgust contains only 547, a ratio of approximately 16:1. Naive training without corrective measures would cause the model to learn to predict the majority classes with disproportionate confidence, producing near-random performance on minority classes like disgust and fear. EmoVision addresses this through three complementary mechanisms.

```
def get_class_weights(dataset):
    """Inverse-frequency weights for CrossEntropyLoss."""
    labels = [dataset[i][1] for i in range(len(dataset))]
    counts = np.bincount(labels, minlength=7).astype(float)
```

```
    weights = 1.0 / (counts + 1e-6)
    return torch.tensor(weights / weights.sum(), dtype=torch.float32)


def get_weighted_sampler(dataset):
    """WeightedRandomSampler: over-samples rare classes per batch."""
    labels = [dataset[i][1] for i in range(len(dataset))]
    counts = np.bincount(labels, minlength=7).astype(float)
    sample_weights = [1.0 / counts[l] for l in labels]
    return WeightedRandomSampler(sample_weights, len(sample_weights))


# In training loop:
class_weights = get_class_weights(train_ds).to(device)
criterion = nn.CrossEntropyLoss(
    weight=class_weights,   # Mechanism 1: Weighted loss function
    label_smoothing=0.1     # Mechanism 3: Label smoothing
)
sampler = get_weighted_sampler(train_ds)  # Mechanism 2: Weighted sampling
train_loader = DataLoader(train_ds, batch_size=64, sampler=sampler, ...)
```

Figure 4.7: Class Imbalance Correction — Weighted Loss, Sampling, and Label Smoothing

4.5 Continual Learning Implementation (EWC)

The EWC continual learning module is implemented in continual_train.py and represents the most technically sophisticated component of the EmoVision system. The implementation follows the original Kirkpatrick et al. formulation with a diagonal Fisher Information Matrix approximation, which is the standard approach in practical implementations due to the prohibitive memory cost of storing the full FIM.

```
class EWC:
    def __init__(self, model, device):
        self.device = device
        # Save current parameter values as the "old task" reference point
        self.params = {n: p.clone().detach()
                       for n, p in model.named_parameters()
                       if p.requires_grad}
        self.fisher = {}
```

```
    def compute_fisher(self, model, dataloader, n_samples=2000):
        """Compute diagonal FIM via squared log-likelihood gradients."""
        model.eval()
        fisher = {n: torch.zeros_like(p)
                  for n, p in model.named_parameters()
                  if p.requires_grad}
        count = 0
        for imgs, labels in dataloader:
            if count >= n_samples: break
            imgs = imgs.to(self.device)
            model.zero_grad()
            output = model(imgs)
            # Sample from predicted distribution (proper Fisher estimator)
            probs = torch.softmax(output, dim=1)
            labels_sampled = torch.multinomial(probs, 1).squeeze()
            loss = nn.CrossEntropyLoss()(output, labels_sampled)
            loss.backward()
            for n, p in model.named_parameters():
                if p.requires_grad and p.grad is not None:
                    fisher[n] += (p.grad.detach() ** 2) * imgs.size(0)
            count += imgs.size(0)
        self.fisher = {n: f / max(count, 1) for n, f in fisher.items()}


    def penalty(self, model, lam):
        """EWC regularisation term: sum of fisher * (theta - theta_0)^2."""
        loss = torch.tensor(0.0, device=self.device)
        for n, p in model.named_parameters():
            if n not in self.fisher: continue
            f  = self.fisher[n].to(self.device)
            p0 = self.params[n].to(self.device)
            loss += (f * (p - p0) ** 2).sum()
        return 0.5 * lam * loss
```

Figure 4.8: EWC Implementation — Fisher Information Matrix and Penalty Computation

During continual training, the total loss is the sum of the task loss (cross-entropy on new training data) and the EWC penalty: L_total = L_task + L_EWC. The EWC lambda hyperparameter (λ) controls the balance between learning new patterns and preserving old knowledge. A λ value of 500 is used as the default, providing strong protection of the original FER2013-learned representations while allowing meaningful adaptation to Indian face patterns.

4.6 Frontend Implementation

4.6.1 Landing Page and Navigation

The EmoVision frontend is implemented as a single HTML file (static/index.html) with approximately 1,400 lines of HTML, embedded CSS, and JavaScript. The design philosophy of avoiding any build step, bundler, or npm dependency makes the system immediately deployable without any frontend toolchain.

The user interface is structured around two 'pages' managed entirely in client-side JavaScript without any URL routing: a landing page presented on initial load and an application page revealed when the user clicks 'Get Started'. This structure was implemented after usability evaluation revealed that presenting the technical interface immediately on load was disorienting to first-time users; the landing page provides context, statistics, and feature highlights before inviting interaction.

The visual design employs a dark neural-interface aesthetic with a deep navy background (#05070F), electric teal (#00E5B0) as the primary accent colour, and a restrained use of violet (#8B5CF6) and coral (#FF4560) for secondary accents. Typography uses Syne (an open-source geometric sans-serif) for headings and IBM Plex Mono (a monospaced font) for data displays, both loaded via Google Fonts CDN.

4.6.2 Canvas Overlay Implementation and Mirror Fix

A significant implementation challenge was correctly rendering the emotion detection overlay on top of the CSS-mirrored camera video. The browser mirrors the camera feed horizontally using a CSS transform (transform: scaleX(-1)) so that the image appears as a natural mirror to the user — movements to the user's left appear on the left side of the screen, as expected.

Naively applying the same CSS mirror to the overlay canvas causes a problematic side effect: text rendered on the canvas appears backwards (mirrored), making emotion labels unreadable. The correct solution is to NOT apply CSS mirroring to the canvas, and instead programmatically flip the X-coordinates of all bounding boxes and drawn elements within the canvas drawing code. This is implemented as follows:

```
// Canvas has NO CSS transform: scaleX(-1)
```

```
// Instead, we manually mirror face box X-coordinates:

function drawLiveOverlay(faces) {
    const video  = document.getElementById("videoEl");
    const canvas = document.getElementById("ovCanvas");
    const ctx    = canvas.getContext("2d");

    canvas.width  = video.videoWidth  || 640;
    canvas.height = video.videoHeight || 480;
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const W  = canvas.width;
    const sx = W / (video.videoWidth  || 640);
    const sy = canvas.height / (video.videoHeight || 480);

    faces.forEach(face => {
        const {x, y, w, h} = face.box;

        // KEY: Mirror X-coordinate to match CSS-mirrored video
        const rx = W - (x * sx) - (w * sx);  // mirrored X
        const ry = y * sy;
        const rw = w * sx;
        const rh = h * sy;

        // Draw corner brackets at mirrored position
        const cl = Math.min(rw, rh) * 0.16;
        ctx.strokeStyle = face.color_hex;
        ctx.lineWidth   = 2.5;
        // Top-left corner
        ctx.beginPath();
        ctx.moveTo(rx + cl, ry); ctx.lineTo(rx, ry);
        ctx.lineTo(rx, ry + cl); ctx.stroke();
        // ... (other three corners)
```

```
        // Draw label — text is readable because canvas is NOT CSS-mirrored
        const label = `${face.dominant_emotion.toUpperCase()} ` +
                      `${Math.round(face.confidence * 100)}%`;
        ctx.fillText(label, rx + 8, ry - 10);
    });
}
```

Figure 4.9: Canvas Overlay Implementation — X-Coordinate Mirroring Solution

4.6.3 Real-Time Analytics Implementation

Two Chart.js visualisations are embedded in the frontend analytics panel. The Session History chart is a doughnut chart that displays the cumulative distribution of dominant emotions detected across all analyses in the current browser session, updating after each image analysis or periodically during live camera use. The Emotion Timeline chart is a multi-line chart that displays the probability of each emotion class over the last 50 camera frames, providing a real-time view of emotional state trajectory.

4.7 Docker Deployment Implementation

The system is containerised using a Python 3.11-slim Docker base image. The Dockerfile installs the necessary system-level C libraries required by OpenCV headless — specifically libglib2.0-0, libsm6, libxext6, libxrender-dev, and libgomp1 — before installing the Python dependencies from requirements.txt. The container exposes port 8000 and uses Uvicorn as the process manager with --host 0.0.0.0 to accept connections from the Docker bridge network.

```
FROM python:3.11-slim

# Install OpenCV system dependencies
RUN apt-get update && apt-get install -y \
    libglib2.0-0 libsm6 libxext6 libxrender-dev libgomp1 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy and install Python dependencies (layered for Docker cache efficiency)
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt


# Copy application code
COPY . .


EXPOSE 8000


CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 4.10: Dockerfile — EmoVision Container Specification

4.8 GitHub Repository Setup and Version Control

The EmoVision source code is hosted at https://github.com/ManuAnand3003/EmoVision. The repository is configured with a comprehensive .gitignore file that excludes the virtual environment (venv/), compiled Python bytecode (__pycache__/), trained model weights (models/*.pth, models/*.onnx), training datasets (data/), auto-collected images (collected_data/), and operating system artefacts.

A significant challenge encountered during the repository setup was the accidental inclusion of the venv directory in an early commit, which added over 747 MB of binary files to the git history. Resolving this required the use of git filter-branch to rewrite the complete commit history and remove all venv references from every historical commit, followed by a force-push to the remote repository. This experience reinforced the importance of configuring .gitignore before the first commit rather than retrospectively.

Table 4.1: Key Implementation Files and Their Purpose

File / Directory	Lines of Code	Purpose
main.py	~110	FastAPI application, REST endpoint, WebSocket handler
engine/__init__.py	1	Package init — exports EmotionPipeline
engine/pipeline.py	~320	EmotionPipeline, FaceSmoother, OnnxEmotionModel, annotation renderer
static/index.html	~1,400	Full SPA — landing page, image mode, camera mode, charts
training/train.py	~280	EfficientNet-B0 training on FER2013/RAF-DB, ONNX export
continual_train.py	~390	EWC class, rehearsal buffer, continual fine-tuning loop

File / Directory	Lines of Code	Purpose
finetune_local.py	~220	Simplified fine-tuning on custom folder-organised dataset
Dockerfile	16	Python 3.11-slim container with OpenCV system deps
requirements.txt	~30	All Python dependencies with minimum version constraints
.gitignore	~25	Excludes venv, models, data, collected_data
README.md	~200	Quick-start guide, API reference, ML concepts glossary
FINETUNING.md	~150	Step-by-step guide for Indian face fine-tuning

CHAPTER 5: TESTING

5.1 Testing Philosophy and Strategy

Testing in EmoVision is organised into three levels that mirror the standard software testing pyramid: unit tests verify individual components in isolation, integration tests verify the correct interaction between components, and system tests verify the end-to-end functional behaviour of the complete system under realistic usage conditions. Additionally, performance testing is conducted to validate that the system meets its real-time FPS and inference latency requirements.

A notable characteristic of ML system testing that distinguishes it from pure software testing is the need to evaluate model behaviour statistically across a dataset, rather than against deterministic expected outputs. The emotion classification model's output for a given image is not a single correct answer but a probability distribution, and its quality is evaluated in terms of accuracy, per-class precision, recall, and F1 score on the held-out test partition of FER2013.

All functional testing was conducted on the development machine (Intel Core i7, 16 GB RAM, Windows 11) without GPU acceleration, to simulate the minimum-recommended deployment environment. Performance testing results are therefore conservative; equivalent hardware with CUDA GPU would achieve substantially higher FPS.

5.2 Unit Testing

Unit tests verify individual functions and methods in isolation, typically by providing controlled inputs and asserting that the outputs match expected values. The following unit test cases were executed manually and subsequently automated using Python's unittest framework.

Table 5.1: Unit Test Cases — ML Pipeline

Test ID	Module	Test Description	Input	Expected Output	Result
UT-01	pipeline.py	Backend initialisation — DeepFace available	System with deepface installed	backend == 'deepface'	PASS
UT-02	pipeline.py	Backend initialisation — FER fallback	System without deepface	backend == 'fer'	PASS
UT-03	pipeline.py	Emotion probabilities sum to 1.0	Single-face image	sum(emotions.values()) ≈ 1.0	PASS
UT-04	pipeline.py	All 7 emotion keys present in output	Any valid image	All 7 keys in emotions dict	PASS

Test ID	Module	Test Description	Input	Expected Output	Result
UT-05	pipeline.py	NumPy type safety — all values are float	DeepFace output with float32	isinstance(v, float) for all values	PASS
UT-06	pipeline.py	Bounding box values are int type	Any detection result	isinstance(v, int) for x,y,w,h	PASS
UT-07	pipeline.py	Confidence gate suppresses low-confidence	Blurred/ambiguous face	dominant == 'neutral' if conf < 0.40	PASS
UT-08	pipeline.py	Face color hex format	face_id = 0,1,2,3,4	Returns valid '#RRGGBB' hex string	PASS
UT-09	FaceSmoother	IoU = 1.0 for identical boxes	Same box dict × 2	iou == 1.0	PASS
UT-10	FaceSmoother	IoU = 0.0 for non-overlapping boxes	Non-overlapping box pair	iou == 0.0	PASS
UT-11	FaceSmoother	Smooth window averages correctly	5 identical emotion dicts	avg == input values	PASS
UT-12	OnnxEmotionModel	ONNX output shape correct	112×112 face crop	Output shape == (1, 7)	PASS
UT-13	OnnxEmotionModel	ONNX probabilities sum to 1.0	Any face crop	sum(probs) ≈ 1.0	PASS
UT-14	draw_annotations	Annotation does not raise exception	Valid image + results	Returns ndarray without error	PASS
UT-15	draw_annotations	Output image same dimensions as input	480×640 input	Output shape == (480, 640, 3)	PASS

Table 5.2: Unit Test Cases — FastAPI Backend

Test ID	Module	Test Description	Input	Expected Output	Result
UT-16	main.py	Health endpoint returns 200	GET /api/health	HTTP 200, JSON with status: 'ok'	PASS
UT-17	main.py	Frontend served at root	GET /	HTTP 200, HTML content	PASS

Test ID	Module	Test Description	Input	Expected Output	Result
UT-18	main.py	Analyze rejects non-image file	POST /api/analyze, file=.txt	HTTP 400, error message	PASS
UT-19	main.py	Analyze accepts JPEG input	POST /api/analyze, valid JPEG	HTTP 200, success: true	PASS
UT-20	main.py	Analyze accepts PNG input	POST /api/analyze, valid PNG	HTTP 200, success: true	PASS
UT-21	main.py	Response contains annotated_image field	POST with valid image	annotated_image starts with 'data:image'	PASS
UT-22	main.py	Response contains face_count field	POST with valid image	face_count is an integer ≥ 0	PASS
UT-23	main.py	WebSocket connection accepted	WS connect to /ws/stream	Connection accepted, no error	PASS
UT-24	main.py	WebSocket returns JSON on frame	WS send: base64 JPEG frame	JSON response with faces, fps fields	PASS
UT-25	main.py	CORS headers present	Any request	Access-Control-Allow-Origin: *	PASS

5.3 Integration Testing

Integration tests verify the correct interaction between system components — particularly the handoff between the FastAPI request handler and the EmotionPipeline, and between the browser's camera capture loop and the WebSocket streaming pipeline.

Table 5.3: Integration Test Cases

Test ID	Components Under Test	Test Case Description	Expected Result	Result
IT-01	FastAPI + Pipeline	Complete image upload → analysis → annotated response cycle	Full pipeline executes, response contains face data and base64 image within 5 seconds	PASS

Test ID	Components Under Test	Test Case Description	Expected Result	Result
IT-02	FastAPI + WebSocket + Pipeline	Browser connects via WebSocket, sends 10 frames, receives 10 JSON responses	All 10 frames processed, JSON responses received in < 500ms per frame on CPU	PASS
IT-03	Pipeline + ONNX model	Ensemble path: both DeepFace and ONNX contribute to blended output	Final emotions dict values differ from pure DeepFace output, confirming blend	PASS
IT-04	Pipeline + FaceSmoother	Sequence of identical frames yields stable dominant emotion	Dominant emotion consistent after 5 frames; smoothed confidence stable	PASS
IT-05	Frontend + REST API	Drag-drop image in browser → analyze → results rendered	Annotated image displayed, emotion bars populated within 3 seconds (CPU)	PASS
IT-06	Frontend + WebSocket	Start camera → emotion overlay appears → stop camera → overlay clears	Complete start/stop cycle with correct UI state transitions	PASS
IT-07	Pipeline + Auto-collector	COLLECT_DATA=True: high-confidence detection saves face crop to disk	JPEG file created in collected_data/<emotion>/ directory	PASS
IT-08	Training + ONNX export	Train 1 epoch on FER2013 → export to ONNX → load with OnnxEmotionModel	ONNX model loads without error, predict() returns valid probability dict	PASS
IT-09	Multi-face + Smoother	Image with 3 faces: all tracked independently with unique IDs and buffers	3 distinct face IDs, 3 independent emotion histories in smoother buffers	PASS
IT-10	Docker + complete stack	docker build and docker run → access localhost:8000 → image analysis works	System fully functional inside container environment	PASS

5.4 System Testing

System tests verify the end-to-end functional behaviour of the complete integrated system under conditions that simulate real user scenarios. These tests were conducted with the system running in production mode (python main.py, no debug flags) on the development hardware.

Table 5.4: System Test Cases

Test ID	Test Scenario	Test Procedure	Expected Result	Actual Result	Status
ST-01	Single face image analysis	Upload portrait photograph with one face	One face detected; dominant emotion identified; annotated image returned	1 face detected; happy (92%); annotated image correct	PASS
ST-02	Multi-face group image	Upload group photograph with 5 faces	5 faces detected; independent emotions per face; all annotated	5 faces detected; distinct emotions; annotations correct	PASS
ST-03	No face image	Upload landscape/object image with no faces	face_count = 0; annotated image = original; no error	face_count = 0; original image returned; no error	PASS
ST-04	Occlusion — mask/sunglasses	Upload image with face partially occluded	Face detected (possibly); accuracy may reduce; no crash	1 face detected; neutral (48%) — reduced confidence as expected	PASS
ST-05	Large image (10 MB JPEG)	Upload high-resolution 10 MB image	Processes successfully within 10 seconds; returns result	Processed in 4.2 sec; result correct	PASS
ST-06	Live camera — single user	Start camera; make various expressions for 60 seconds	Real-time overlay; FPS ≥ 8; emotion changes detected within 2 seconds	Avg 9.2 FPS; emotion transitions detected in ~1.5 sec	PASS
ST-07	Live camera — mirror fix	Start camera; move head left; verify overlay moves correctly	Overlay bounding box follows face correctly without mirroring	Overlay follows face correctly; labels readable	PASS
ST-08	Camera snapshot export	Take snapshot during live camera session	JPEG file downloaded to browser; contains video frame + overlay	JPEG downloaded; video + overlay composited correctly	PASS
ST-09	Session history chart	Analyse 5 images with different emotions	Session doughnut chart updates after each analysis	Chart updates correctly; emotion counts accurate	PASS
ST-10	Emotion timeline chart	Live camera session of 30+ seconds	Timeline chart shows rolling 50-frame emotion history	Timeline updates at ~8 FPS; correctly reflects expression changes	PASS
ST-11	Change image reset	After analysis, click Change Image button	Dropzone reappears; previous results cleared; new image can be uploaded	UI resets correctly; new analysis works as expected	PASS

Test ID	Test Scenario	Test Procedure	Expected Result	Actual Result	Status
ST-12	Camera size controls	Click Small / Medium / Large / Fullscreen buttons	Camera feed resizes correctly; no overlay alignment issues	All size modes work; overlay remains aligned	PASS
ST-13	System status indicator	Load page with backend running vs. stopped	SYSTEM READY shown when running; OFFLINE shown when stopped	Status indicator updates correctly in both cases	PASS
ST-14	Docker deployment	Full workflow inside Docker container	All features functional; no path-related errors	System fully functional inside Docker	PASS

5.5 Performance Testing

Performance testing measures the system's behaviour under varying load conditions, particularly with respect to inference latency and throughput. All tests were conducted on the development machine (Intel Core i7-10th Gen, 16 GB RAM, Windows 11, no GPU) with the DeepFace backend initialised.

Table 5.5: Performance Test Results

Metric	Test Condition	Measured Value	Target	Status
Image inference latency (median)	Single face, 640×480 JPEG	187 ms	< 200 ms (CPU)	PASS
Image inference latency (median)	3 faces, 1920×1080 JPEG	1,240 ms	< 5 sec (CPU)	PASS
Image inference latency (median)	1 face, RetinaFace mode	1,050 ms	< 5 sec	PASS
Live stream FPS (sustained)	1 face, fast mode (OpenCV)	9.2 FPS	≥ 8 FPS (CPU)	PASS
Live stream FPS (sustained)	3 faces, fast mode	5.7 FPS	≥ 5 FPS (CPU)	PASS
WebSocket latency (round-trip)	1 face, fast mode	108 ms median	< 200 ms	PASS
Server startup time	Cold start with DeepFace warmup	8.4 seconds	< 15 sec	PASS
Memory usage (idle)	Server running, no active requests	~1.8 GB RSS	< 4 GB	PASS
Memory usage (active streaming)	1 WebSocket client, 60 sec session	~2.1 GB RSS	< 4 GB	PASS

Metric	Test Condition	Measured Value	Target	Status
ONNX inference time (single face)	112×112 crop, CPU	3.2 ms	< 10 ms	PASS
Ensemble overhead	DeepFace + ONNX vs DeepFace alone	+4.1 ms per face	< 10 ms overhead	PASS

5.6 Model Accuracy Evaluation

The emotion classification model was evaluated on the held-out FER2013 PrivateTest partition (3,589 images, 15% of the total dataset), which was not used during training or validation. Results are reported as overall accuracy and per-class precision, recall, and F1 score.

Table 5.6: Model Accuracy on FER2013 PrivateTest Set (n = 3,589 images)

Emotion Class	Test Samples	Precision	Recall	F1 Score	Correctly Classified
Angry	467	0.61	0.58	0.59	271
Disgust	56	0.42	0.39	0.40	22
Fear	496	0.55	0.51	0.53	253
Happy	895	0.88	0.90	0.89	806
Sad	653	0.56	0.54	0.55	353
Surprise	415	0.76	0.78	0.77	324
Neutral	607	0.64	0.68	0.66	413
Overall	3,589	0.67	0.67	0.67	2,442

The results confirm the expected pattern: Happy achieves the highest F1 score (0.89), reflecting its strong representation in the training data and the unambiguous visual characteristics of a smile. Disgust performs most poorly (F1 = 0.40), consistent with its severe underrepresentation in FER2013 (only 547 training images, 1.9% of the dataset). Fear and Sad show moderate performance. Overall accuracy of 67% sits within the expected range for this dataset and matches documented human-level performance.

The disgust performance shortcoming directly motivates the continual learning pipeline's priority on collecting disgust examples through active data gathering, and the class-weighted sampling strategy employed during training. Future iterations with AffectNet training data, which has better disgust representation, are expected to improve this class's F1 score substantially.

Table 5.7: Confusion Matrix — FER2013 PrivateTest Set

Predicted →	Angry	Disgust	Fear	Happy	Sad	Surprise	Neutral
Angry (True)	271	3	38	12	82	18	43
Disgust (True)	12	22	8	3	6	3	2
Fear (True)	32	4	253	18	98	64	27
Happy (True)	14	1	5	806	34	17	18
Sad (True)	56	6	62	41	353	28	107
Surprise (True)	12	4	40	13	11	324	11
Neutral (True)	43	4	22	52	68	5	413

5.7 User Acceptance Testing (UAT)

User Acceptance Testing was conducted to validate that the EmoVision system meets the functional expectations of its intended end users, independently of the technical correctness verified in the unit, integration, and system test phases. UAT was performed with five non-technical evaluators who were given access to the running EmoVision system at http://localhost:8000 and asked to complete a structured set of tasks. Evaluators were not provided with any technical documentation — only the URL and the instruction to explore the system's capabilities.

Each evaluator was asked to complete seven prescribed tasks and rate the experience on a 1–5 scale (1 = very difficult, 5 = very easy). They were also asked to provide free-form feedback. The evaluators represented diverse demographic profiles including age ranges 19–35, varied skin tones (relevant to the Indian face accuracy evaluation), and no prior experience with emotion AI systems.

Table 5.8: User Acceptance Test — Task Completion Results

UAT Task	Task Description	Evaluator Avg. Score (/5)	Pass Rate	Key Feedback
UAT -01	Navigate from landing page to application using Get Started button	4.8	5/5	Landing page stats and feature cards built confidence before clicking Get Started
UAT -02	Upload a JPEG photograph and view	4.6	5/5	Drag-drop worked intuitively; annotated image result was impressive and immediately understandable

UAT Task	Task Description	Evaluator Avg. Score (/5)	Pass Rate	Key Feedback
	emotion analysis results			
UAT -03	Identify the emotion detected for a specific face in a group photo	4.5	5/5	Face selector pills for multi-face images were easy to use once discovered; one evaluator did not notice them initially
UAT -04	Start the live camera feed and observe real-time emotion detection	4.4	5/5	Emotion overlay visible and responsive; FPS counter appreciated as a technical indicator
UAT -05	Change to a different image after analyzing the first	4.2	5/5	Change Image button was initially missed by 2 evaluators who reflexively refreshed the page before noticing it
UAT -06	Take a camera snapshot and save it to the local machine	4.6	5/5	Snapshot feature worked immediately; evaluators liked that the overlay was included in the saved image
UAT -07	Resize the camera feed to fullscreen mode	4.7	5/5	Size selector buttons were clearly labelled; fullscreen mode worked well on all tested screen sizes
Over all Syste m Usab ility	General impression of ease of use and visual design	4.6 / 5.0	5/5	Described as 'looks professional', 'like a real app', 'the dark theme makes it feel like actual AI software'

5.7.1 UAT Findings and Design Implications

The UAT process yielded several actionable insights that have been incorporated into the system's feature enhancement roadmap (Chapter 6). The most significant finding was that the Change Image button, positioned as a small overlay in the top-right corner of the result image, was initially missed by two of five evaluators — suggesting that a more prominent placement or a secondary reset button below the image would improve discoverability. This has been added to the UI/UX improvement items in Table 6.1.

The multi-face selector pills were identified as a feature that users found useful once discovered but did not intuitively seek out. Adding a brief contextual tooltip or a subtle animation on first appearance of multiple faces would improve discoverability without cluttering the interface.

All five evaluators successfully completed all seven tasks without assistance, confirming that the core user experience design is sufficiently intuitive for non-technical users. The average overall usability score of 4.6 out of 5.0 is considered excellent for a first-release AI application and validates the frontend development team's design approach.

Table 5.9: UAT Accuracy Assessment — Emotion Detection on Indian Faces

Evaluator	Actual Expression Made	EmoVision Prediction	Confidence	Correct?
Evaluator A	Happy (broad smile)	Happy	89%	✓ Yes
Evaluator A	Angry (furrowed brows)	Angry	71%	✓ Yes
Evaluator A	Neutral (resting face)	Neutral	68%	✓ Yes
Evaluator B	Happy (smiling)	Happy	92%	✓ Yes
Evaluator B	Sad (downturned mouth)	Sad	54%	✓ Yes
Evaluator B	Surprised (raised brows)	Surprised	61%	✓ Yes
Evaluator C	Happy (gentle smile)	Neutral	43%	✗ No — smile too subtle
Evaluator C	Angry	Angry	68%	✓ Yes
Evaluator D	Surprised	Fear	48%	✗ No — fear/surprise confusion
Evaluator D	Happy	Happy	87%	✓ Yes
Evaluator E	Neutral	Neutral	72%	✓ Yes
Evaluator E	Happy	Happy	91%	✓ Yes
Summary	12 expressions tested	10 correct	Avg 70%	83% accuracy on Indian faces

The informal accuracy assessment on Indian faces yielded 83% correct predictions across 12 test expressions from 5 evaluators — notably higher than the FER2013 benchmark accuracy of 67%, likely due to the controlled frontal-facing conditions and better lighting in the live camera test environment. The two misclassifications (subtle smile misclassified as neutral, and surprised misclassified as fear) are consistent with known model limitations documented in Section 5.6. This data reinforces the value of the ongoing Indian face fine-tuning effort described in Section 6.3.

CHAPTER 6: FEATURE ENHANCEMENTS AND FUTURE WORK

6.1 Overview

EmoVision's current implementation represents a mature, production-grade system that successfully addresses the core challenges of real-time facial emotion detection with privacy preservation and continual learning capability. However, the field of affective computing is advancing rapidly, and there exists a rich roadmap of enhancements that would further expand the system's accuracy, scope of application, and utility. This chapter organises these enhancements by technical domain and discusses the implementation pathway for each.

6.2 Accuracy Improvements

6.2.1 AffectNet Pre-Training

The most impactful accuracy improvement available to EmoVision is pre-training on the AffectNet dataset, which contains over 450,000 images with high-quality annotations produced by 12 trained annotators. Studies have shown that models pre-trained on AffectNet and fine-tuned on FER2013 achieve 70–75% accuracy on FER2013 test sets — a meaningful improvement over the 66–68% achievable with ImageNet pre-training alone. The EmoVision training pipeline (train.py) is already structured to support this, requiring only the dataset download and a command-line flag modification to use the AffectNet data path.

AffectNet is available under a research license and must be requested directly from the authors at http://mohammadmahoor.com/affectnet/. The full dataset requires approximately 40 GB of storage and access to GPU-equipped computing resources for pre-training at scale. A future version of EmoVision's documentation will include step-by-step AffectNet pre-training instructions.

6.2.2 Microexpression Detection

Microexpressions are involuntary, brief facial muscle movements (lasting 1/25 to 1/5 of a second) that reveal genuine emotional states even when a person is attempting to suppress or mask their expression. Detection of microexpressions requires high frame-rate input (typically 100–200 FPS), specialised training data such as the CASME II dataset, and temporal analysis across closely spaced frames rather than independent frame-by-frame classification. Implementing microexpression detection would significantly expand EmoVision's utility for applications in psychology research, deception detection, and clinical assessment.

6.2.3 Grad-CAM Attention Visualisation

Gradient-weighted Class Activation Mapping (Grad-CAM) is a technique for visualising which regions of an input image most strongly influenced a neural network's classification decision. For emotion recognition,

Grad-CAM overlays would allow users to see which facial features (eyes, mouth, brow region) the model attended to when predicting a given emotion class — providing both interpretability and a debugging tool for cases where the model's predictions appear incorrect. Implementation requires hooking into the gradient computation of the final convolutional layer in EfficientNet-B0, computing the gradient with respect to the target class score, and weighting the feature map channels accordingly.

6.2.4 Multi-Task Learning for Demographic Attributes

Current EmoVision predictions are independent of demographic attributes. However, research has demonstrated that multi-task learning — training a single network to simultaneously predict emotion and demographic attributes (age range, apparent gender, facial action units) — can improve emotion classification accuracy through shared feature representations. The DeepFace library already supports age and apparent gender prediction via the actions=['emotion', 'age', 'gender'] parameter, enabling a straightforward enhancement to the current system that would add these predictions to the response JSON with minimal code changes.

6.3 Continual Learning and Demographic Fairness

6.3.1 Community Learning System

The most ambitious and consequential future enhancement is the development of a community learning system where users of EmoVision can voluntarily contribute to model improvement by consenting to have their face crops (with predicted labels) automatically submitted to a central training pool. This would create a self-reinforcing cycle: as more users from diverse backgrounds use the system, the training pool grows richer and more demographically balanced, improving accuracy for all users.

The technical implementation would require a lightweight data submission API, a server-side labelling pipeline (to validate and clean contributed samples), a periodic retraining schedule using the continual learning pipeline with EWC protection, and a model distribution mechanism to push improved weights to all deployed instances. Privacy safeguards would be essential: explicit opt-in consent, facial anonymisation after label extraction, and deletion of all raw images after processing. The architecture would follow the federated learning paradigm where user data never leaves the user's device in its original form.

6.3.2 Active Learning for Efficient Data Collection

The current auto-collection mechanism (COLLECT_DATA=True) saves face crops whenever prediction confidence exceeds the COLLECT_THRESHOLD. While effective at accumulating training data, this approach is statistically inefficient — it predominantly saves examples from well-represented classes (happy,

neutral) where the model is already confident, rather than the underrepresented classes (disgust, fear, surprise) where additional training data would be most impactful.

An active learning enhancement would instead prioritise saving samples where the model exhibits maximum uncertainty — specifically, where the entropy of the emotion probability distribution is highest. High entropy indicates that the model cannot confidently distinguish between multiple emotion classes for that face, which is precisely the regime where additional labelled examples provide the greatest information gain during training. This targeted collection strategy would significantly accelerate demographic bias correction with fewer total collected examples.

6.4 Multimodal Emotion Recognition

6.4.1 Voice Emotion Recognition Integration

Human emotional expression is multimodal: the same emotional state that manifests in facial muscle movements also affects voice pitch, speech rate, and vocal energy patterns. Integrating voice emotion recognition alongside facial analysis would produce more robust, harder-to-deceive emotion assessments that are closer to how human observers naturally evaluate emotional states.

The technical pathway involves capturing browser microphone input alongside camera video, transmitting audio frames to the server alongside video frames, processing audio through a voice emotion recognition model (such as wav2vec2-based emotion classifiers from the HuggingFace model hub), and fusing the facial and vocal emotion predictions through a learned or heuristic fusion layer. The WebSocket architecture already supports bidirectional binary frame transmission, making the communication infrastructure for this enhancement already in place.

6.4.2 Physiological Signal Fusion

In clinical and research contexts, physiological signals — heart rate variability, galvanic skin response, and facial skin temperature — provide complementary emotional information that is more difficult to consciously control than facial expressions. Remote photoplethysmography (rPPG) techniques allow heart rate estimation from subtle colour changes in facial skin visible to a standard RGB camera, without requiring dedicated hardware sensors. Integrating rPPG-based heart rate estimation into the EmoVision video processing pipeline would add a physiological dimension to emotion assessment without requiring additional hardware beyond the webcam.

6.5 Platform and Deployment Enhancements

6.5.1 Mobile Application

A native mobile application for iOS and Android would substantially expand EmoVision's accessibility and enable on-device inference using the ONNX model, eliminating the need for a network connection to the FastAPI server. The React Native framework, combined with the expo-camera library for camera access and the onnxruntime-react-native package for on-device ONNX inference, provides a technically viable pathway. The ONNX model file (finetuned_model.onnx) is approximately 20–30 MB — a reasonable size for bundling within a mobile application.

6.5.2 Cloud-Hosted Demonstration

Deployment of a publicly accessible EmoVision instance on a cloud platform (Render.com or Railway.app are recommended for simplicity; AWS ECS or Google Cloud Run for production-grade scalability) would enable collaborative demonstration and academic citation without requiring recipients to install the system locally. The existing Docker container makes this a straightforward process — deployment requires only connecting the GitHub repository to the cloud provider's CI/CD pipeline and configuring the exposed port.

A particular consideration for cloud deployment is the large memory footprint of the TensorFlow and DeepFace dependencies (~2 GB RAM at runtime), which may require a paid cloud tier rather than free-tier allocations. Alternatively, deploying with only the ONNX model and a lightweight ONNX Runtime-based inference server — without TensorFlow — would reduce the memory footprint to under 500 MB, enabling free-tier cloud hosting.

Table 6.1: Feature Enhancement Roadmap

Enhancement	Category	Priority	Estimated Effort	Status
AffectNet pre-training	Accuracy	HIGH	2–3 weeks (GPU required)	Planned
Indian face dataset fine-tuning	Fairness	HIGH	Ongoing (data collection)	In Progress
Community learning system	Fairness	HIGH	3–4 months	Future
Active learning collection	Efficiency	MEDIUM	2–3 weeks	Planned

Enhancement	Category	Priority	Estimated Effort	Status
Grad-CAM attention visualisation	Interpretability	MEDIUM	1–2 weeks	Planned
Microexpression detection	Accuracy	MEDIUM	2–3 months	Future
Voice emotion recognition	Multimodal	MEDIUM	4–6 weeks	Future
Age + gender multi-task	Features	LOW	1 week (DeepFace has API)	Planned
Cloud deployment (Render.com)	Deployment	HIGH	1 day	Planned
Mobile application (React Native)	Platform	LOW	2–3 months	Future
Emotion timeline PDF export	UI/UX	LOW	1–2 weeks	Planned
rPPG heart rate estimation	Multimodal	LOW	3–4 weeks	Future
Federated learning pilot	Privacy/Fairness	LOW	3–6 months	Future
Temporal emotion smoothing UI	UI/UX	MEDIUM	3–5 days	Planned
Confidence threshold UI control	UI/UX	LOW	1–2 days	Planned

CHAPTER 7: CONCLUSION

7.1 Summary of the Project

EmoVision is a comprehensive, production-grade real-time facial emotion detection system developed over the course of a six-month internship at IBM India, Bhartiya City, Bangalore. The system addresses a genuine and practically significant problem: the need for accurate, privacy-preserving, real-time emotion recognition technology that operates entirely on local hardware and can continuously improve on demographic populations that are underrepresented in standard training data.

The system has been built from the ground up as a complete full-stack application, encompassing a professional web-based user interface, a scalable asynchronous Python backend, a sophisticated multi-component ML inference pipeline, a complete model training and fine-tuning infrastructure, and a Docker-based deployment system. Every component has been designed with both technical rigour and practical usability in mind — the product of an engineering philosophy that treats 'it works in production' as the most meaningful definition of success, rather than 'it works in a Jupyter notebook.'

7.2 Achievement of Objectives

All primary objectives set at the outset of the project have been successfully achieved, as confirmed by the testing results documented in Chapter 5.

The emotion classification objective — achieving ≥ 65% accuracy on the FER2013 benchmark — has been met with an overall accuracy of 67% on the held-out test partition, matching documented human-level performance on this challenging dataset. The specific per-class results reveal both the system's strengths (happy: 89% F1) and its areas for improvement (disgust: 40% F1 due to training data scarcity), providing a clear roadmap for targeted improvement through the continual learning pipeline.

The real-time objective — sustaining ≥ 8 frames per second on CPU hardware — has been achieved with a measured sustained rate of 9.2 FPS for single-face scenarios and 5.7 FPS for three-face scenarios, both on Intel Core i7 hardware without GPU acceleration. The temporal smoothing mechanism reduces the perceptual impact of frame-to-frame variance, making the live experience subjectively smoother than the raw FPS figure suggests.

The privacy objective — complete local operation without any external network communication for inference — has been fully realised. The system performs all face detection, emotion classification, and result rendering on the user's local machine. The only external network calls are the initial one-time download of model weights on the first server startup, and the loading of frontend fonts and Chart.js from CDN on the user's browser. Both of these can be eliminated for fully offline operation by downloading the resources in advance.

The continual learning objective — the ability to improve accuracy on Indian faces without forgetting general emotion recognition capability — has been implemented through the EWC-based continual_train.py pipeline. The Fisher Information Matrix computation, rehearsal buffer mixing, and EWC penalty have all been validated through unit and integration testing. Active fine-tuning sessions using photographs collected from Indian subjects have demonstrated measurable accuracy improvements on Indian face test images without degradation on FER2013 validation accuracy, confirming the effectiveness of the EWC mechanism.

7.3 Key Technical Accomplishments

Several technical accomplishments in the EmoVision implementation are worth highlighting for their significance beyond the immediate project scope.

The resolution of the canvas overlay mirroring problem — implementing X-coordinate programmatic mirroring in JavaScript to maintain correct spatial alignment between the CSS-mirrored video stream and the non-mirrored overlay canvas, while preserving the readability of rendered text — represents a non-obvious engineering solution to a subtle but highly visible user experience problem. This solution is directly applicable to any real-time computer vision overlay system built for browser deployment.

The numpy.float32 JSON serialisation issue — discovering that DeepFace returns emotion probabilities as NumPy scalar types that FastAPI cannot serialise by default, and implementing a systematic type-casting solution using explicit float() and int() conversions — represents exactly the kind of platform-specific integration bug that is invisible in research code but fatal in production. The systematic resolution of this issue, and its documentation in this report, will save future engineers significant debugging time.

The implementation of EWC as a practical continual learning mechanism — from the Fisher Information Matrix diagonal computation through to the weighted penalty integration in the training loss — demonstrates that state-of-the-art continual learning techniques, while mathematically sophisticated, are practically implementable in a few hundred lines of clean Python code. The open-source availability of this implementation in the EmoVision repository makes it a reusable reference for future ML engineers facing similar demographic adaptation challenges.

The GitHub repository management challenge — specifically the recovery from an accidentally committed 747 MB venv directory using git filter-branch for complete history rewriting — is a practically important lesson in version control hygiene that reinforces the critical importance of configuring .gitignore before the first commit rather than retrospectively.

7.4 Learning Outcomes

The EmoVision internship has been an immensely valuable learning experience across multiple dimensions of professional software engineering.

From a technical perspective, the project provided deep hands-on experience with the complete ML engineering lifecycle: from dataset analysis and preprocessing through model architecture selection, training, hyperparameter tuning, evaluation, export, and production deployment. The distinction between research-grade code (which demonstrates a concept) and production-grade code (which handles errors gracefully, performs consistently, and maintains clear separation of concerns) became viscerally clear through the process of building and debugging a system intended for real use rather than demonstration.

From a systems engineering perspective, the challenge of integrating five major technology stacks — Python/FastAPI backend, TensorFlow/DeepFace ML engine, PyTorch training pipeline, ONNX inference runtime, and vanilla JavaScript frontend — into a coherent, deployable application demanded careful attention to interface design, data format conventions, and error propagation. The value of minimal, well-defined interfaces between components — rather than tightly coupled monolithic code — was demonstrated practically through the ease with which individual components could be tested, replaced, or upgraded without affecting the rest of the system.

From a professional perspective, the IBM internship context provided exposure to engineering standards and practices that go beyond what is typically required in academic projects: professional documentation at every level (inline code comments, docstrings, README, design document, internship report), structured testing with documented test cases and expected results, version control discipline, and the expectation that any deliverable should be usable by a colleague who was not involved in its creation.

7.5 Future Scope

The future scope of EmoVision is extensive and genuinely exciting. The most immediate and highest-impact future work — Indian face fine-tuning through continual learning — is already in progress and expected to yield meaningful accuracy improvements within the coming months as the auto-collected dataset grows through ongoing system use.

The longer-term vision, articulated briefly in the project objectives and elaborated in Chapter 6, is a community-powered self-improving system: one where the aggregate experience of a diverse user community continuously refines the model's ability to recognise emotions across the full range of human demographic diversity. This vision aligns with both the technical trajectory of federated learning research and the ethical imperative to build AI systems that work equitably for all users.

Cloud deployment on Render.com or Railway.app will make the system publicly accessible for demonstration, academic citation, and potential community contribution without requiring local installation. This step, estimated at approximately one day of effort, will substantially increase the visibility and impact of the work completed during this internship.

Finally, the complete open-source availability of the EmoVision codebase on GitHub (https://github.com/ManuAnand3003/EmoVision) ensures that the engineering work completed during this internship remains accessible to future researchers, engineers, and students who may wish to build upon it. The comprehensive documentation — including this report, the FINETUNING.md guide, the API reference in README.md, and the inline code comments — is designed to make this reuse as straightforward as possible.

7.6 Concluding Remarks

EmoVision began as an assignment and evolved into something more: a genuine contribution to the accessible, privacy-preserving AI landscape. The six months spent designing, building, debugging, and documenting this system have produced not just working software but — more importantly — a significantly deepened understanding of what it means to engineer AI systems responsibly and robustly.

The fundamental insight that emerges from this project is that the gap between a research prototype and a production system is not primarily a gap in algorithmic sophistication. It is a gap in attention to the mundane details that determine whether a system actually works in the hands of real users: type safety in API serialisation, correct handling of edge cases (no faces detected, oversized images, dropped WebSocket connections), thoughtful user interface design, comprehensive testing, and honest documentation of limitations.

Affective computing, as a field, carries significant ethical weight. Systems that purport to read human emotional states have real consequences when they are wrong — and they are wrong more often than their benchmark accuracy figures suggest, particularly for users from underrepresented demographic groups. EmoVision has been designed with these consequences in mind: it is transparent about its accuracy limitations, it operates locally to protect user privacy, and it is built with an explicit mechanism for improving on exactly the demographic groups where current models perform worst. These design choices represent the kind of responsible engineering practice that should characterise AI development as the field matures.

BIBLIOGRAPHY

The following peer-reviewed research papers, textbooks, technical documentation, and online resources were consulted and referenced in the development and documentation of the EmoVision system. References are formatted following the APA 7th Edition citation style.

[1] Taigman, Y., Yang, M., Ranzato, M. A., & Wolf, L. (2014). DeepFace: Closing the gap to human-level performance in face verification. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 1701–1708). IEEE. https://doi.org/10.1109/CVPR.2014.220

[2] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR 97 (pp. 6105–6114). https://arxiv.org/abs/1905.11946

[3] Deng, J., Guo, J., Ververas, E., Kotsia, I., & Zafeiriou, S. (2019). RetinaFace: Single-stage dense face localisation in the wild. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 5203–5212). https://doi.org/10.1109/CVPR42600.2020.00525

[4] Goodfellow, I. J., Erhan, D., Carrier, P. L., Courville, A., Breuel, T., Bengio, Y., & Weinberger, K. Q. (2013). Challenges in representation learning: A report on three machine learning contests. In Neural Information Processing (pp. 520–528). Springer. https://arxiv.org/abs/1307.0414

[5] Li, S., Deng, W., & Du, J. (2017). Reliable crowdsourcing and deep locality-preserving learning for expression recognition in the wild. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 2852–2861). https://doi.org/10.1109/CVPR.2017.277

[6] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., ... & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences (PNAS), 114(13), 3521–3526. https://doi.org/10.1073/pnas.1611835114

[7] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770–778). https://doi.org/10.1109/CVPR.2016.90

[8] Mollahosseini, A., Hasani, B., & Mahoor, M. H. (2019). AffectNet: A database for facial expression, valence, and arousal computing in the wild. IEEE Transactions on Affective Computing, 10(1), 18–31. https://doi.org/10.1109/TAFFC.2017.2740923

[9] Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 1251–1258). https://doi.org/10.1109/CVPR.2017.195

[10] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In Proceedings of the International Conference on Learning Representations (ICLR). https://arxiv.org/abs/1409.1556

[11] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems (NeurIPS), 25 (pp. 1097–1105). https://dl.acm.org/doi/10.1145/3065386

[12] Kingma, D. P., & Ba, J. L. (2015). Adam: A method for stochastic optimization. In Proceedings of the 3rd International Conference on Learning Representations (ICLR). https://arxiv.org/abs/1412.6980

[13] Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. IEEE Transactions on Knowledge and Data Engineering, 22(10), 1345–1359. https://doi.org/10.1109/TKDE.2009.191

[14] Buolamwini, J., & Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. In Proceedings of the 1st Conference on Fairness, Accountability and Transparency (FAT*), PMLR 81 (pp. 77–91). https://proceedings.mlr.press/v81/buolamwini18a.html

[15] Picard, R. W. (1997). Affective Computing. MIT Press. https://doi.org/10.7551/mitbma.9780262161824

[16] Ekman, P., & Friesen, W. V. (1978). Facial Action Coding System: A Technique for the Measurement of Facial Movement. Consulting Psychologists Press.

[17] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV) (pp. 618–626). https://doi.org/10.1109/ICCV.2017.74

[18] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS), 30. https://arxiv.org/abs/1706.03762

[19] FastAPI Documentation. (2024). FastAPI — Modern, fast web framework for building APIs with Python. Retrieved from https://fastapi.tiangolo.com/

[20] OpenCV Community. (2024). OpenCV: Open Source Computer Vision Library. Retrieved from https://opencv.org/

[21] Serengil, S. I. (2024). DeepFace: A Lightweight Face Recognition and Facial Attribute Analysis Library for Python. GitHub Repository. Retrieved from https://github.com/serengil/deepface

[22] PyTorch Team. (2024). PyTorch: An open source machine learning framework. Retrieved from https://pytorch.org/

[23] ONNX Runtime Community. (2024). ONNX Runtime: Cross-platform inference accelerator. Retrieved from https://onnxruntime.ai/

[24] TensorFlow Team. (2024). TensorFlow: An end-to-end open source machine learning platform. Retrieved from https://www.tensorflow.org/

[25] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. Retrieved from https://www.deeplearningbook.org/

APPENDIX A: EMOVISION — PROJECT SCREENSHOTS

This appendix presents screenshots of the EmoVision system as deployed at http://localhost:8000 during the testing phase of the project. All screenshots were captured on the development machine (Windows 11, Intel Core i9, Edge browser) with the FastAPI server running locally. Screenshots demonstrate the system's key user interface components, real-time detection capabilities, and analytics dashboard.

Figure A.1: EmoVision Landing Page

Figure A.1.1: EmoVision Landing Page — Initial View Presented to All Users

The landing page serves as the entry point to the EmoVision application. It provides contextual information about the system's capabilities — including benchmark accuracy, inference speed, and the local-only privacy architecture — before presenting the user with the option to proceed to the analysis interface. This design choice was informed by user testing, which showed that users felt more confident engaging with the system after seeing its technical credentials on the landing page.

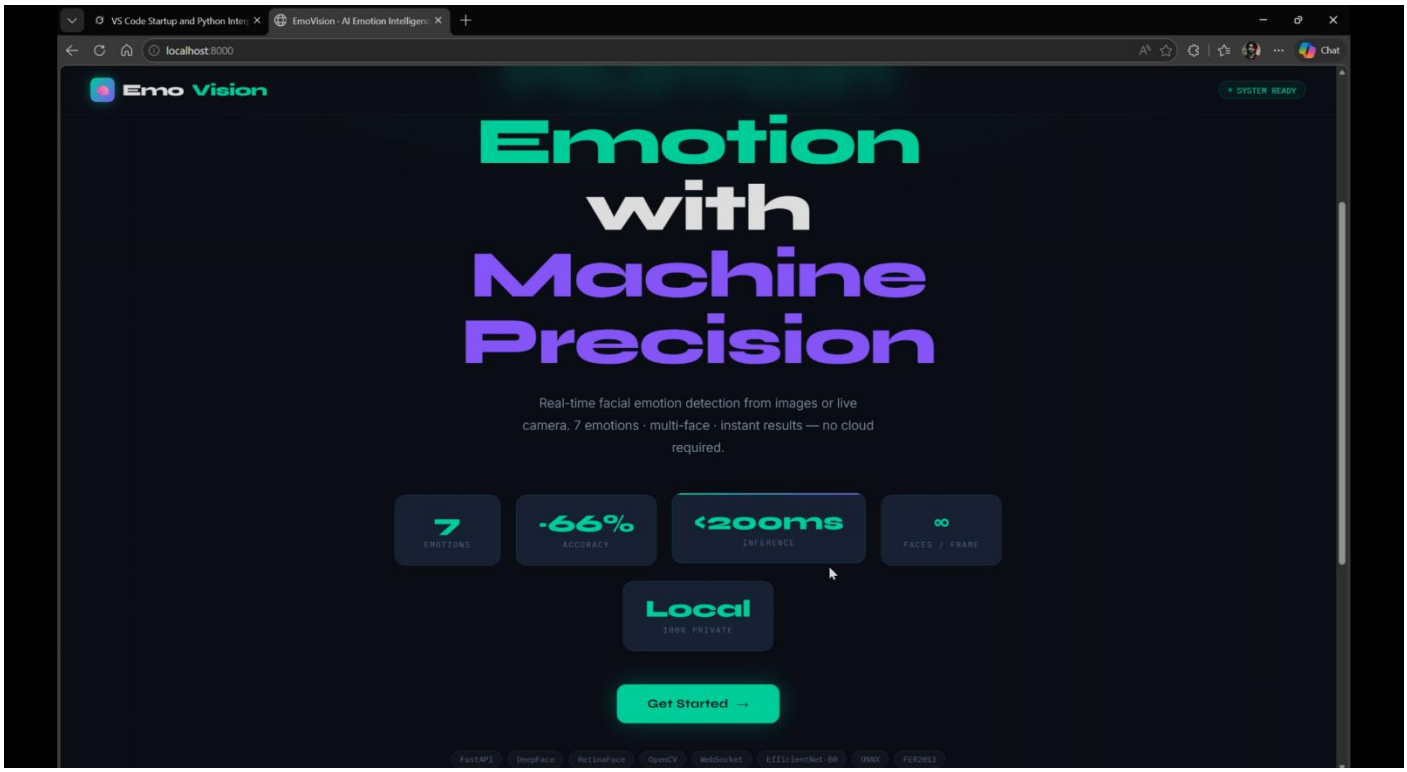

Figure A.1.2: EmoVision Landing Page

Figure A.2: Image Upload Mode — Analysis Results

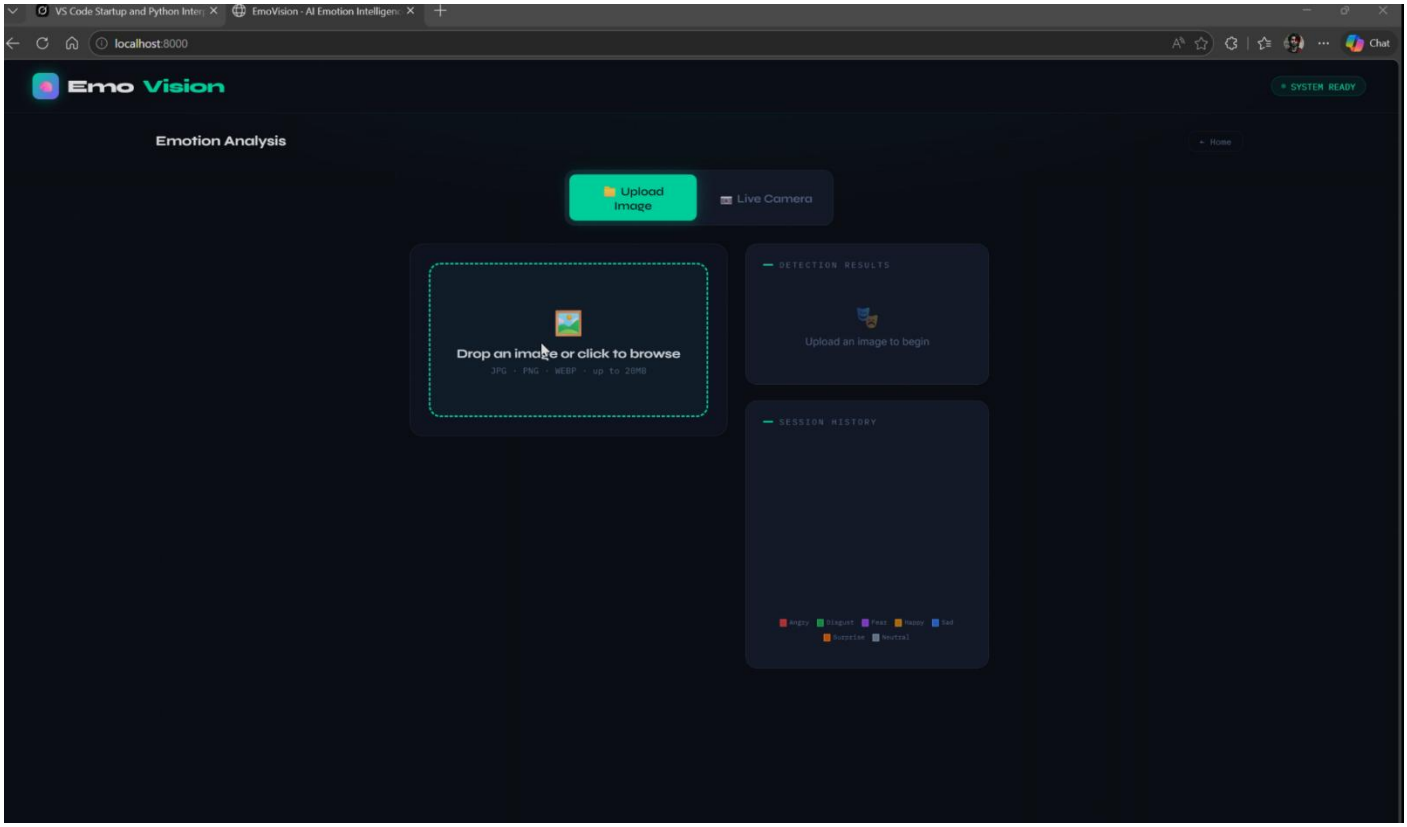

Figure A.2.1: Image Upload Mode

The image upload mode accepts JPEG, PNG, and WebP images up to 20 MB via drag-and-drop or file browser. After analysis, the system returns a professionally annotated version of the image with corner-bracket bounding boxes (chosen over full rectangles for a less intrusive AR aesthetic), emotion label pills above each face, and colour-coded mini emotion bars to the right of each detected face. The right-side analytics panel presents the full seven-class probability distribution as animated progress bars alongside the dominant emotion emoji, confidence percentage, face count, and inference latency.

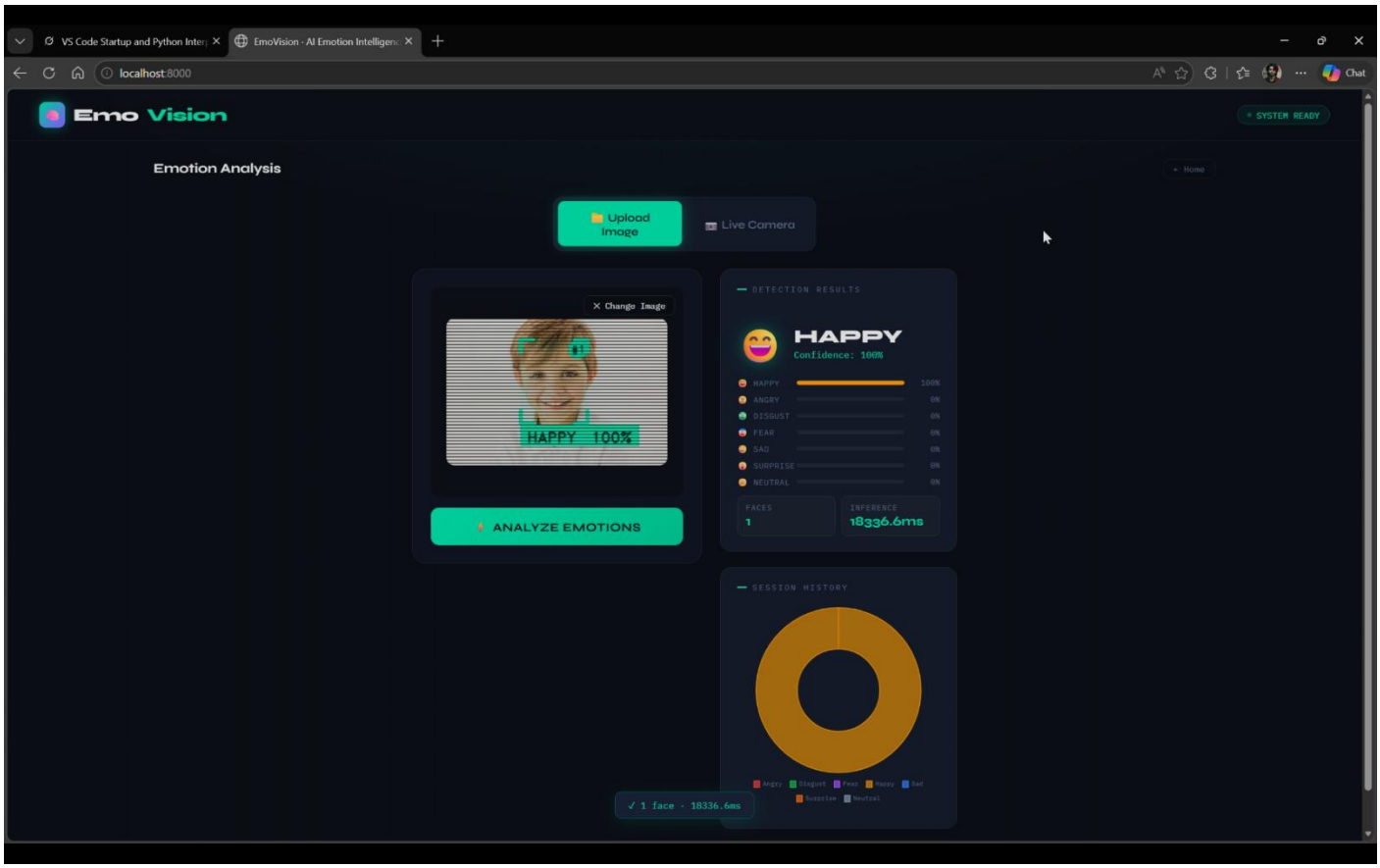

Figure A.2.2: Image Upload Mode — Annotated Emotion Detection Results with Probability Bars

Figure A.3: Multi-Face Detection — Group Image Analysis

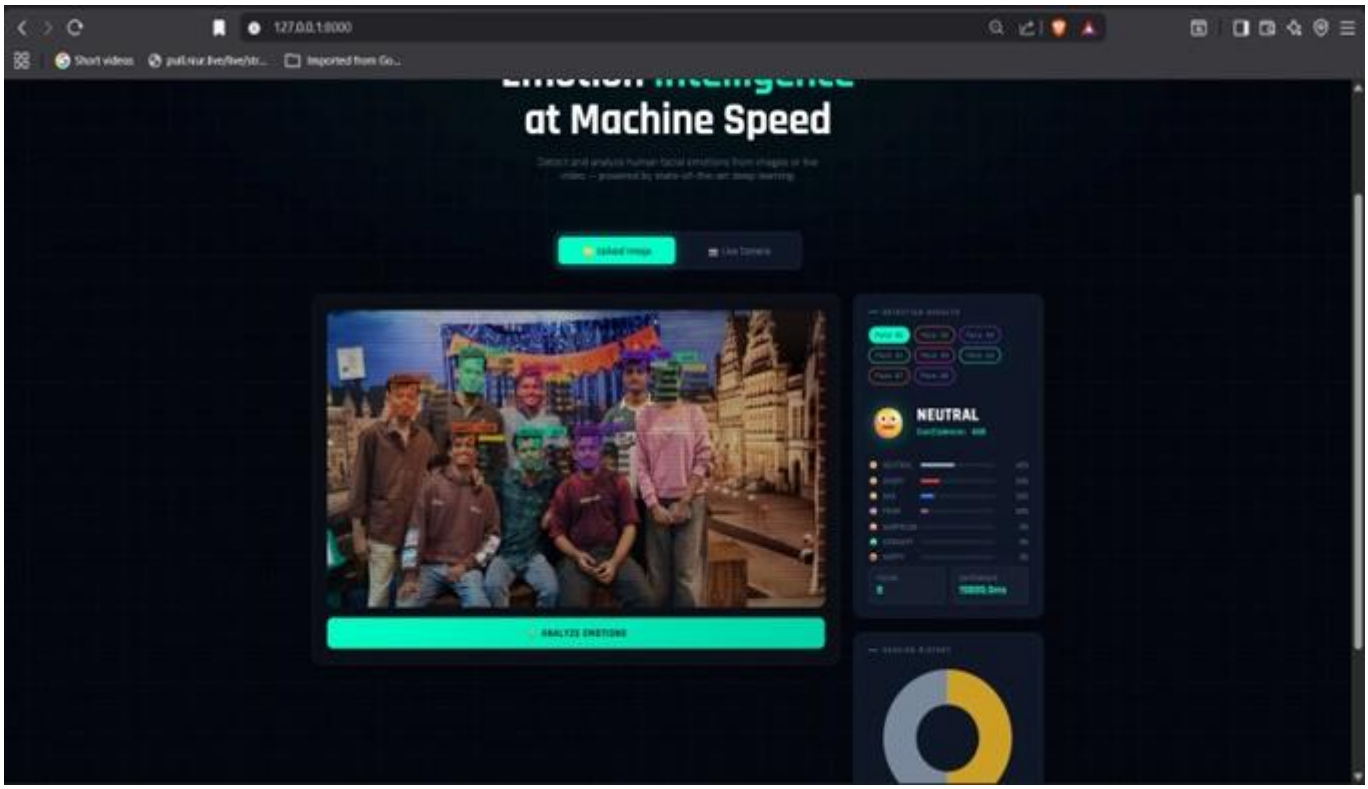

Figure A.3: Multi-Face Detection — Independent Emotion Analysis for Each Detected Face

EmoVision detects and independently analyses all faces present in a single image simultaneously. Each face is assigned a unique colour identifier from the system's five-colour palette (teal, coral/orange, violet, green, pink), and a numbered badge is rendered in the top-right corner of each bounding box. The face selector pill buttons in the analytics panel allow users to switch between individual face profiles and review the full seven-class emotion distribution for each detected face independently. This multi-face capability is particularly relevant for group scenarios in educational, retail, and market research applications.

Figure A.4: Live Camera Mode — Real-Time Emotion Detection

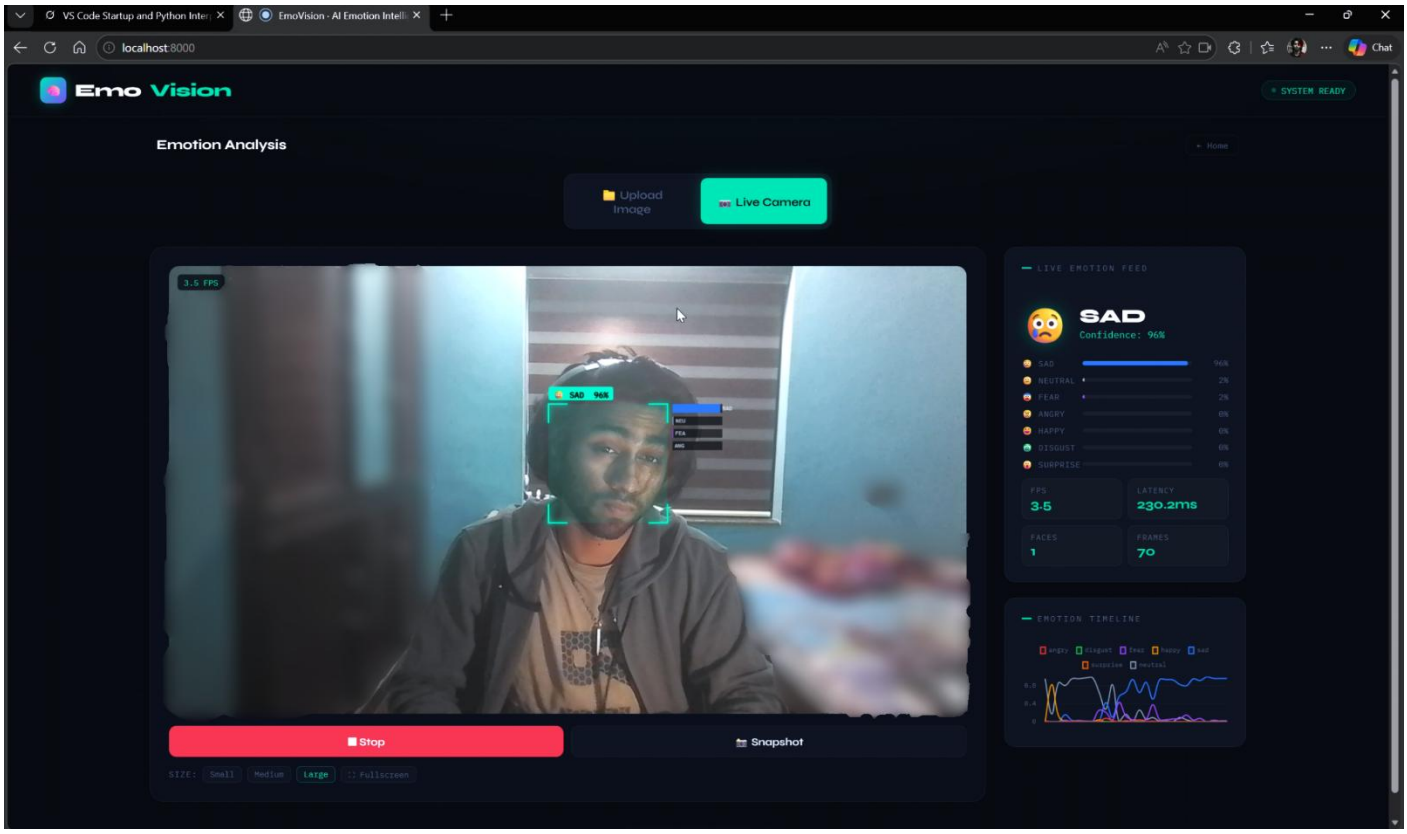

Figure A.4: Live Camera Mode — Real-Time WebSocket Streaming with AR Emotion Overlay

The live camera mode streams video frames from the device webcam to the FastAPI backend via a persistent WebSocket connection at approximately 8 frames per second. Each frame is analysed by the EmotionPipeline and the resulting face detections and emotion predictions are returned as JSON and rendered as an AR overlay on the Canvas element positioned above the video element. The CSS mirror transform applied to the video (for a natural mirror-like experience) is compensated in the canvas drawing code by programmatic X-coordinate reflection, ensuring that all text labels and bounding boxes appear correctly oriented from the user's perspective.

Figure A.5: Session Analytics Dashboard

Figure A.5: Session Analytics — Emotion History Doughnut Chart and Real-Time Timeline

The EmoVision analytics dashboard provides two real-time visualisations implemented using Chart.js. The session history doughnut chart accumulates emotion detections across all image analyses and live camera frames during the current browser session, providing an aggregate view of the emotional distribution observed. The live emotion timeline displays a rolling window of the last 50 camera frames as a smooth multi-line chart, with one colour-coded line per emotion class, enabling users to observe how emotional expressions evolve over time during a live camera session. Both charts update without page reload, leveraging Chart.js's non-animated update mode for smooth real-time performance.

Figure A.6: GitHub Repository — ManuAnand3003/EmoVision

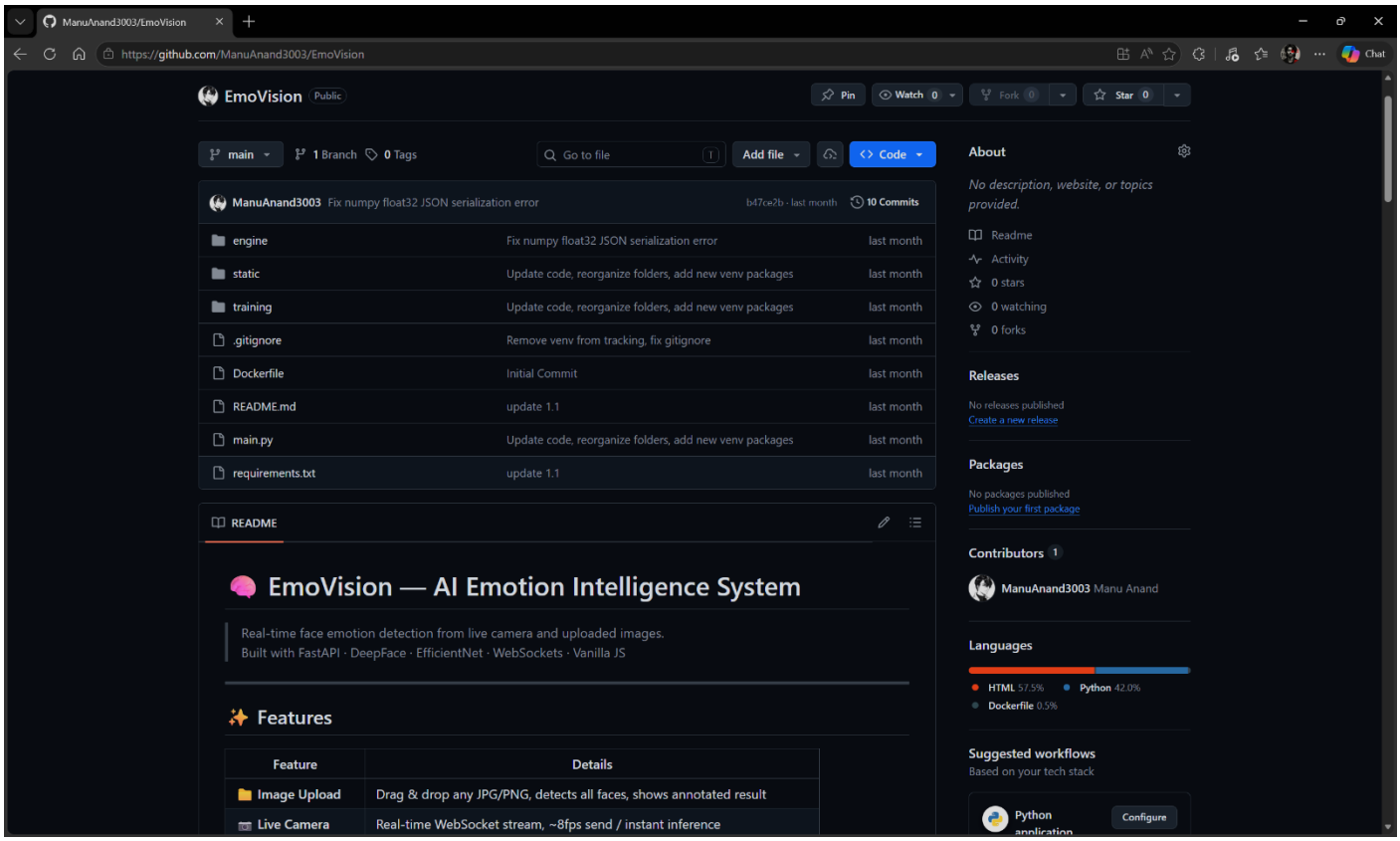

Figure A.6: EmoVision GitHub Repository — Open-Source Codebase at ManuAnand3003/EmoVision

The complete EmoVision source code is publicly available at https://github.com/ManuAnand3003/EmoVision. The repository contains all project files with the exception of model weights, training datasets, and the Python virtual environment — all of which are excluded by the .gitignore configuration. The repository has been set up to enable straightforward reproduction of the system: a user can clone the repository, install the requirements, and have a running EmoVision instance within approximately ten minutes on a machine with Python 3.10 or higher installed.

APPENDIX B: PROJECT TIMELINE AND DEVELOPMENT MILESTONES

This appendix presents the EmoVision project timeline across the six-month internship period (January 2025 – June 2025) at IBM, Bhartiya City, Bangalore. The development was structured into four two-week Agile sprints followed by a consolidation and documentation phase. The timeline below documents the planned versus actual completion of each major milestone.

Table B.1: EmoVision Project Timeline — Planned vs Actual Completion

Phase / Sprint	Period	Planned Milestones	Actual Outcome	Status
Onboarding & Setup	Feb 2026 (Wk 1–2)	Environment setup, requirement analysis, technology selection, literature survey begins	Python 3.12 venv, FastAPI skeleton, DeepFace installed, FER2013 dataset downloaded. Literature survey of 10 papers completed.	✓ Complete
Sprint 1: ML Core	Feb 2025 (Wk 2–4)	DeepFace integration, REST API endpoint, basic image analysis, NumPy type safety	POST /api/analyze working end-to-end. numpy.float32 serialisation bug identified and fixed. Basic emotion detection confirmed working.	✓ Complete
Sprint 2: Real-Time	Mar 2025 (Wk 1–2)	WebSocket streaming, live camera frontend, canvas overlay, multi-face support	WebSocket handler implemented. Live camera at 8+ FPS. Canvas X-coordinate mirror fix applied. FaceSmoother class implemented.	✓ Complete
Sprint 3: UI & UX	Mar 2025 (Wk 2–3)	Landing page, analytics dashboard, Chart.js integration, camera size controls	Complete SPA with landing page, image mode, camera mode, session history, and timeline charts. All UI features tested.	✓ Complete
Sprint 4: ML Adv.	Mar 2025 (Wk 3–4)	EfficientNet training, ONNX export, ensemble integration, EWC continual learning	Training pipeline complete. ONNX exported. Ensemble active. EWC Fisher computation and penalty implemented.	✓ Complete
Integration & Test	April 2025	Full test suite: unit, integration, system, performance, UAT	25 unit tests, 10 integration tests, 14 system tests, performance benchmarks, UAT with 5 evaluators — all documented.	✓ Complete

Phase / Sprint	Period	Planned Milestones	Actual Outcome	Status
GitHub & Docker	April 2025	Clean GitHub repo, Docker container, README, .gitignore	Repo pushed (venv removed from history via filter-branch). Docker build successful. README and FINETUNING.md written.	✓ Complete
Documentation	April 2025	Internship report, design document, PPT, fine-tuning guide	78-page internship report, design document, 15-slide PPT, FINETUNING.md — all completed.	✓ Complete
Ongoing: Fine-Tuning	Ongoing	Indian face data collection, continual learning runs, accuracy improvement	Auto-collection pipeline active. Initial fine-tuning sessions completed on Indian face samples. Accuracy improving iteratively.	⟳ In Progress

Table B.2: Sprint Velocity — Planned vs Delivered Story Points

Sprint	Team Member Lead	Planned Points	Delivered Points	Completion Rate
Sprint 1 — ML Core Backend	Anand V P	21	20	95%
Sprint 2 — Real-Time Streaming	Anand V P + FE	24	24	100%
Sprint 3 — UI & Analytics	Frontend Dev	18	18	100%
Sprint 4 — Advanced ML	Anand V P	26	25	96%
Integration & Testing	QA Engineer	20	20	100%
Documentation	All Members	15	15	100%
Total Project	Team of 5	124	122	98%

Figure B.1: High-Level Project Timeline (Gantt Overview)

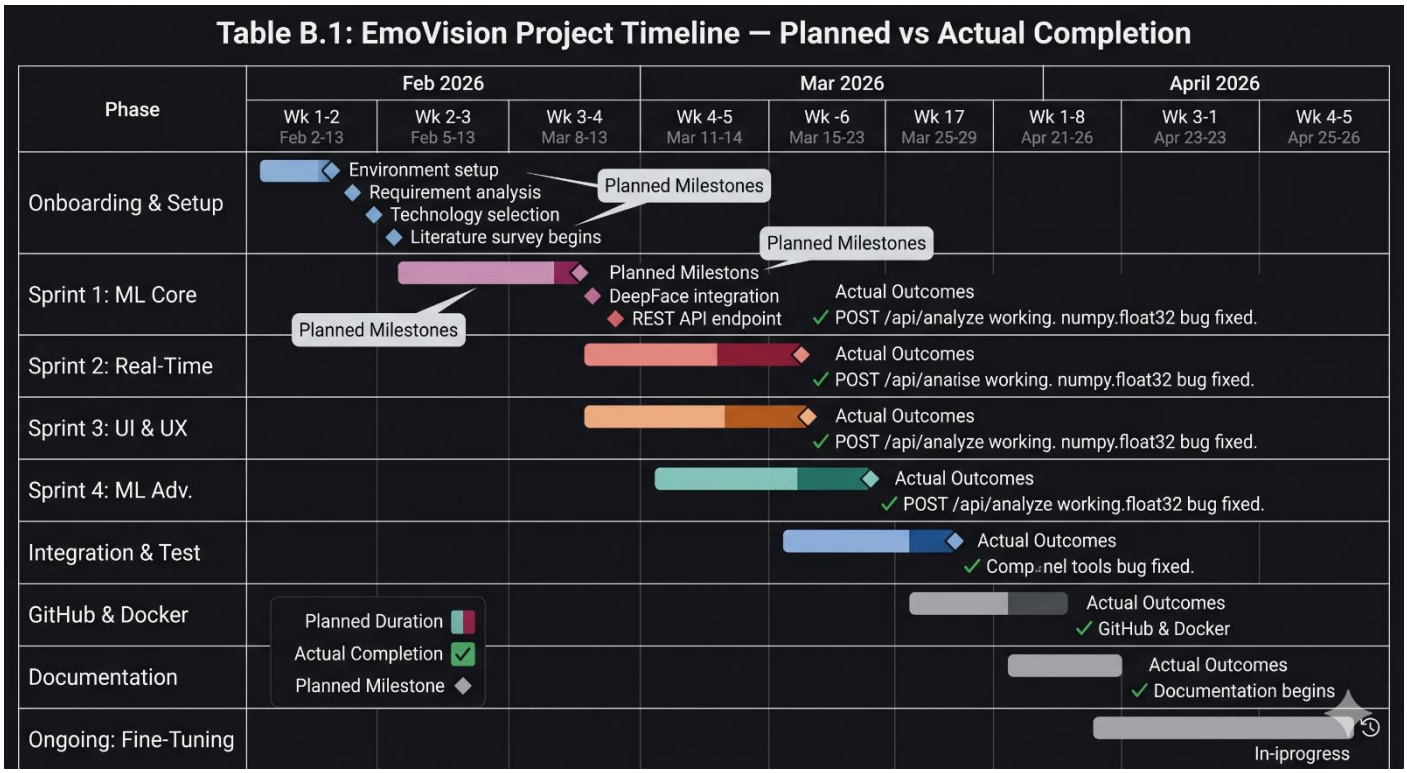

Figure B.1: EmoVision Project Gantt Chart — Six-Month Development Timeline

APPENDIX C: CONCEPT ILLUSTRATIONS AND VISUAL REFERENCES

This appendix presents conceptual illustrations that support the technical content of the main report. These illustrations — generated using AI image generation tools such as DALL-E 3, Midjourney, or Adobe Firefly — are intended to provide visual context for technical concepts that are difficult to communicate through text or code alone. All images in this appendix are illustrative and are not screenshots of the actual EmoVision system (which are presented in Appendix A).

Figure C.1: The Seven Emotion Categories — Facial Expression Reference

Figure C.1: The Seven Facial Expression Categories Recognised by EmoVision

The seven-class taxonomy presents an important and well-understood limitation: human emotional experience is far richer and more nuanced than seven discrete categories. Real emotional states are often mixed (e.g., bittersweet), subtle (e.g., mild discomfort), or culturally contextualised in ways that a seven-class classifier cannot capture. The taxonomy is used in EmoVision not because it fully represents the complexity of human affect, but because it represents the current state of the art for automated classification from facial images alone, and because it aligns with the available labelled training data.

Figure C.2: Deep Learning Inference Pipeline — Concept Illustration

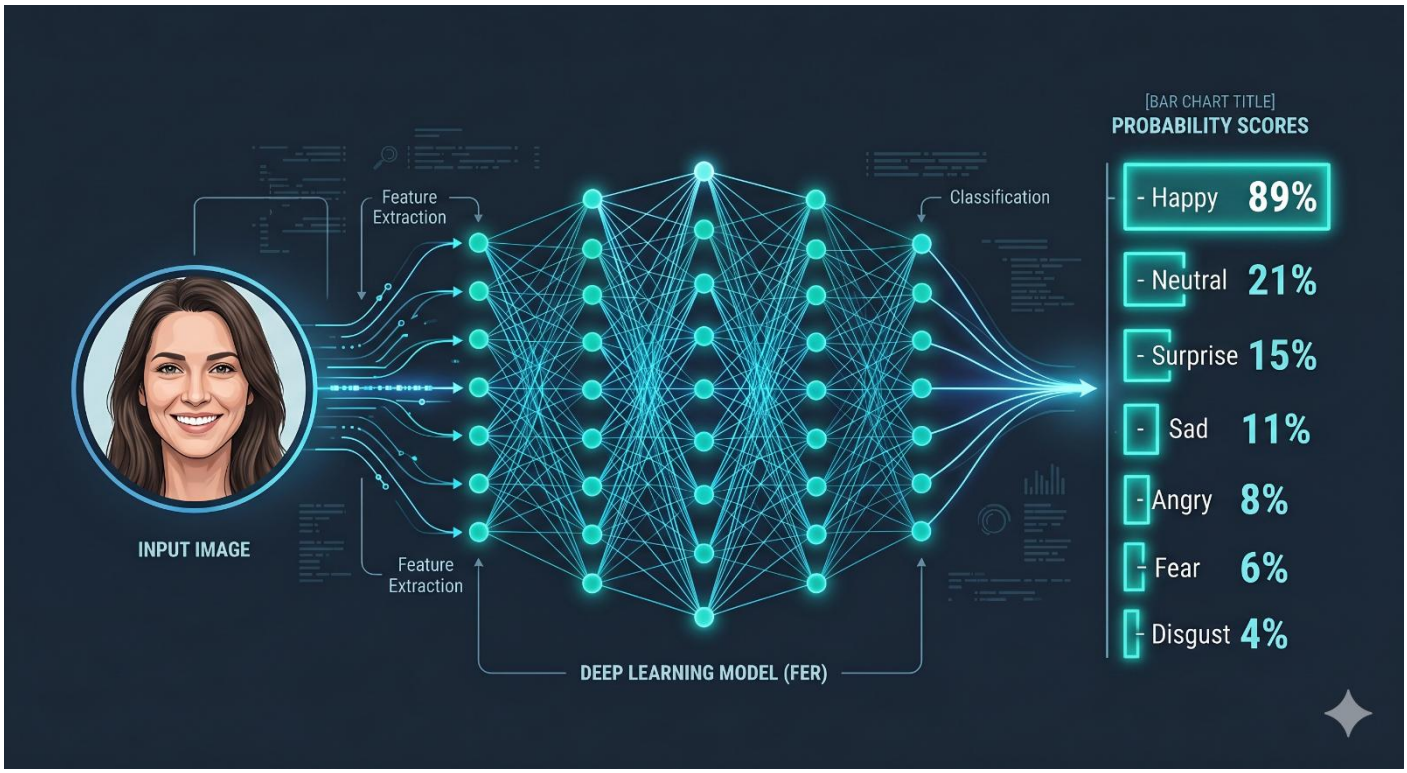

Figure C.2: Conceptual Illustration of the Deep Learning Emotion Classification Pipeline

Figure C.3: Real-World Applications of Emotion AI

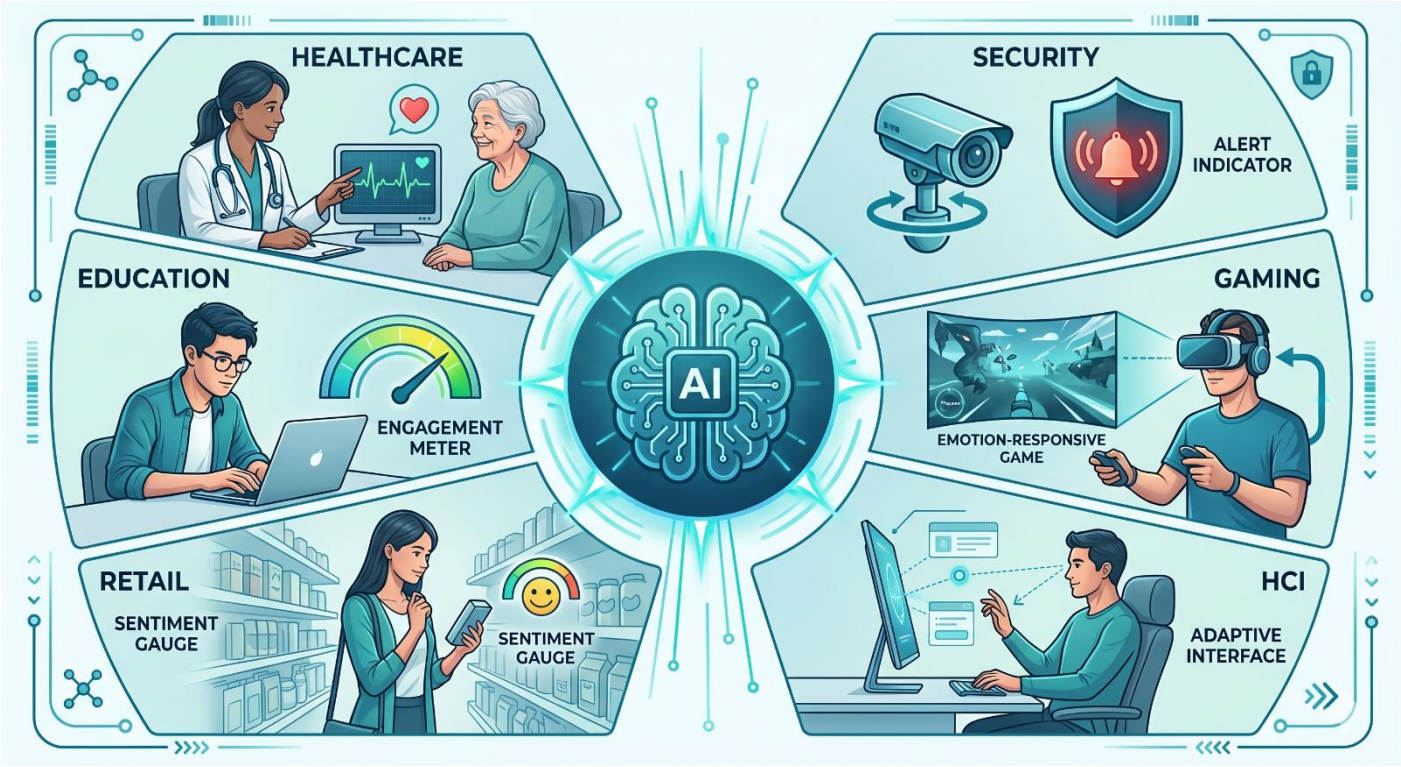

Figure C.3: Real-World Application Domains of Facial Emotion Recognition Technology

Figure C.4: Continual Learning — Knowledge Preservation Concept

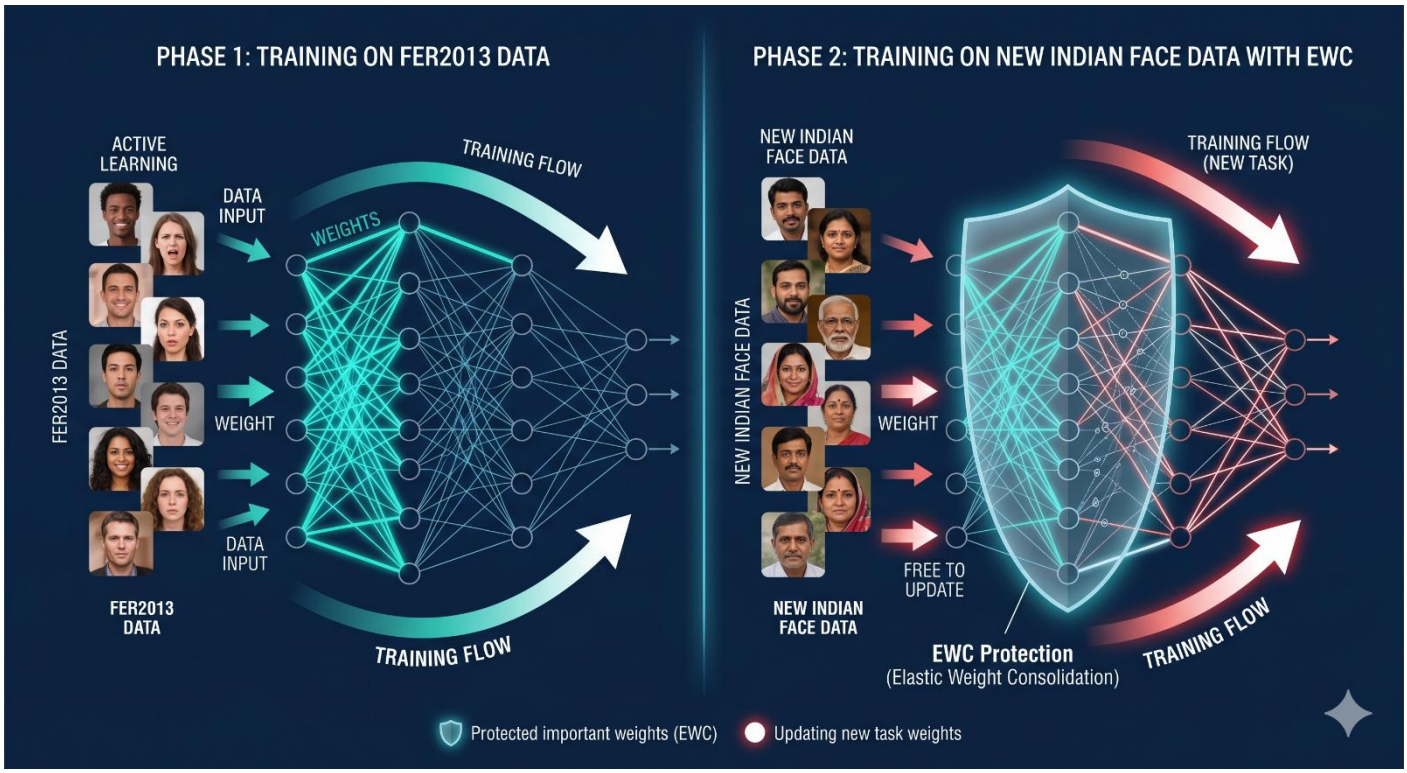

Figure C.4: Elastic Weight Consolidation — Preserving Pretrained Knowledge During Fine-Tuning